

EECS 836: Machine Learning

Zijun Yao

Assistant Professor, EECS Department
The University of Kansas

Agenda

- Convolutional Neural Networks (CNN)
 - What is CNN
 - Convolutional Layer
 - Pooling Layer
- Recurrent Neural Networks (RNN)
 - What is RNN
 - Computation of RNN
 - Language Modeling
 - Long Short Term Memory (LSTM) Networks

Stable Diffusion

- **Prompt:** A beautifully composed magical world, emotional and smiling girl, half body shot, environmental candid of a magic water and fire particles, with captivating, detailed eyes, natural imperfections. taken in a rainforest.



Computer vision



Self-driving vehicles

Computer vision



Face ID

Image recognition

airplane



automobile



bird



cat



deer



dog



frog



horse



ship



truck



[CIFAR-10 tiny image dataset with 10 classes](#)

The MNIST Dataset

label = 5



label = 0



label = 4



label = 2



label = 1



label = 3



label = 3



label = 5

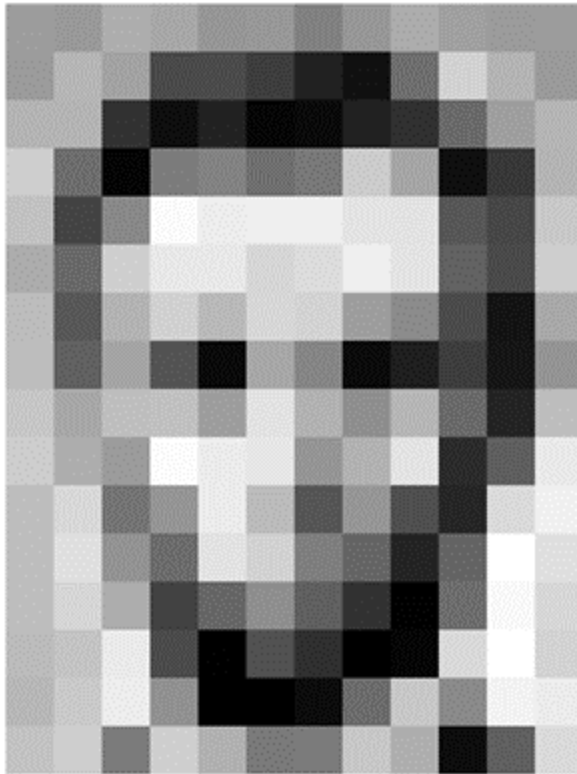


label = 3



[MNIST handwritten digit Dataset](#)

Images are matrices of numbers

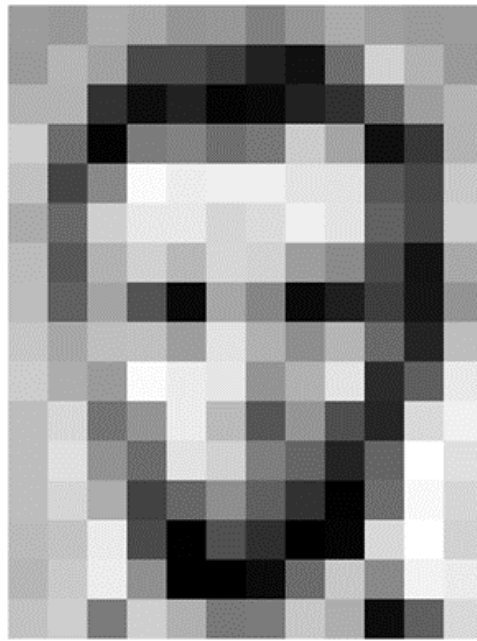


157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

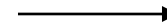
157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

An image is a matrix of numbers $[0, 255]$ (color intensity)

Classification of image



157	153	174	168	150	152	129	151	172	161	155	156
156	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	228	227	87	71	201	
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218



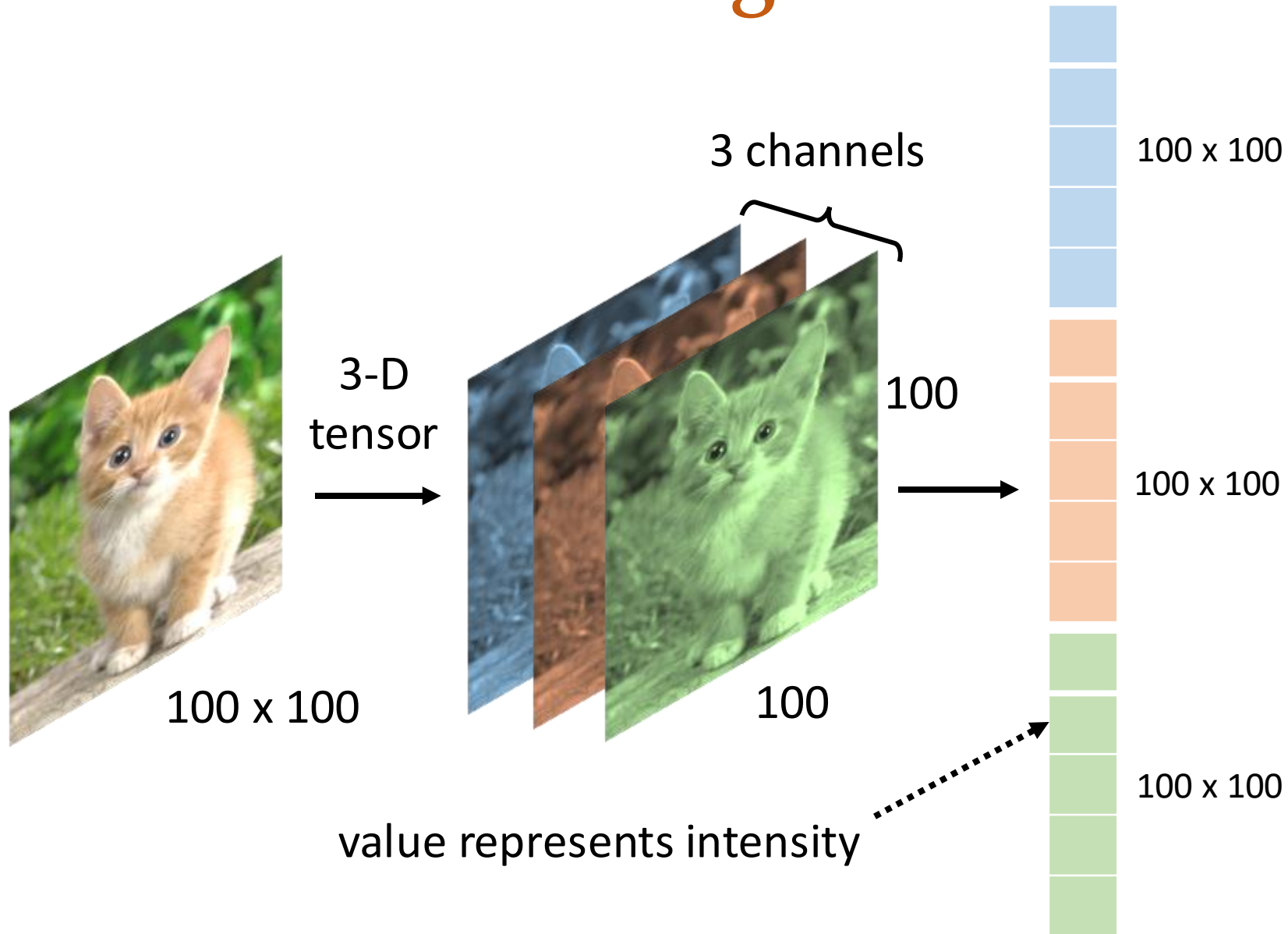
Lincoln	0.8
Washington	0.1
Jefferson	0.05
Obama	0.05

y $\longleftrightarrow$ $\hat{y}$

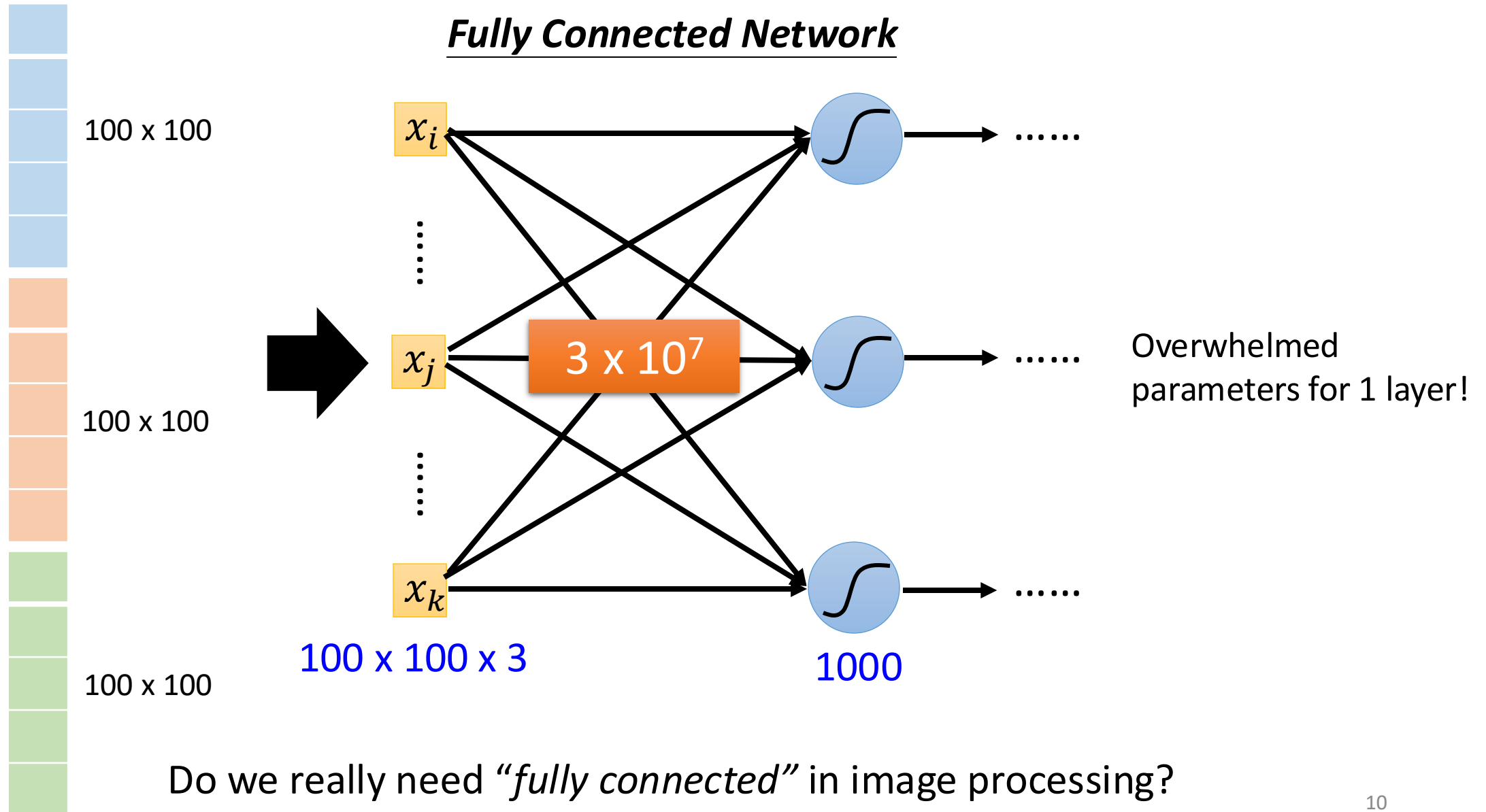
Cross entropy

Classification: predict probability of belonging to a particular class

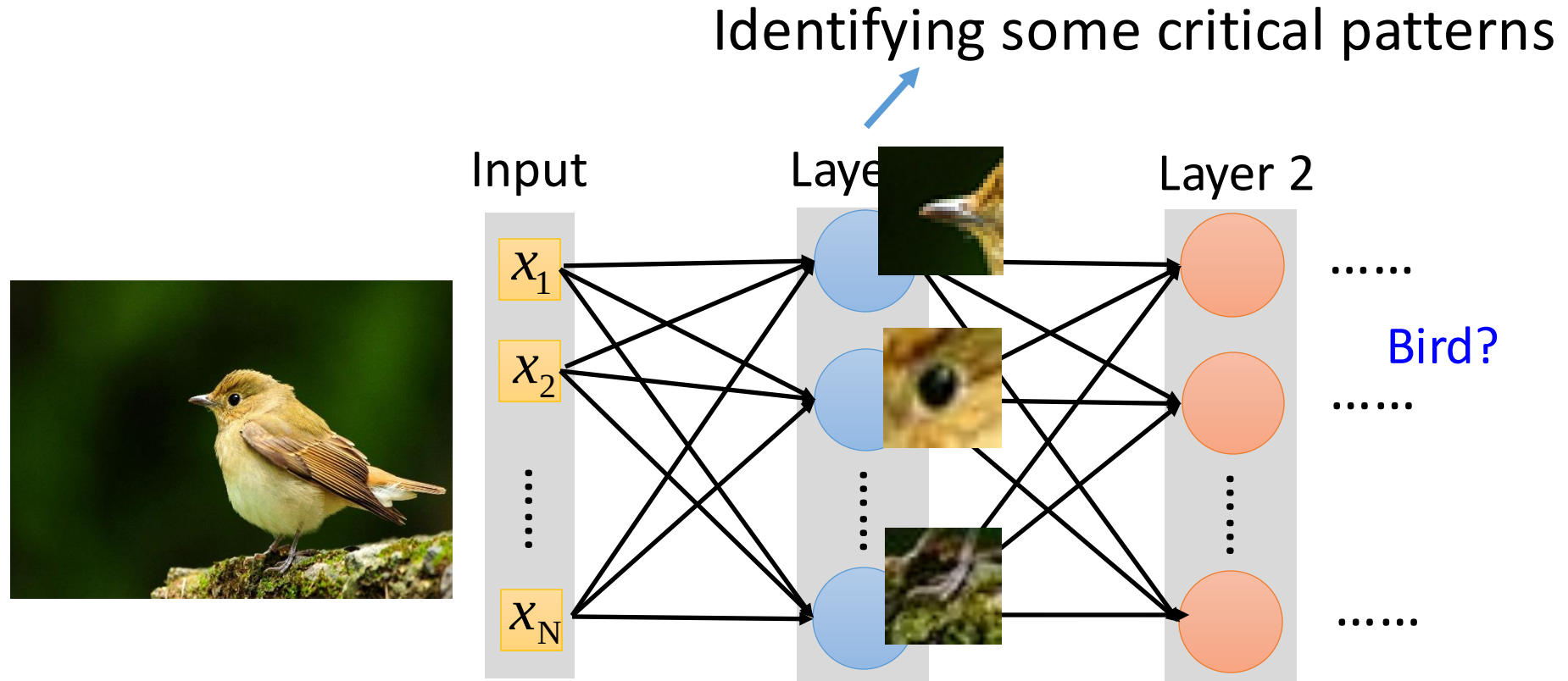
Classification of with regular NN



Classification of with regular NN



Observation



Our human don't need all the pixels to identify a bird

Why CNN for image learning (1/3)

- Some patterns are much smaller than the whole image

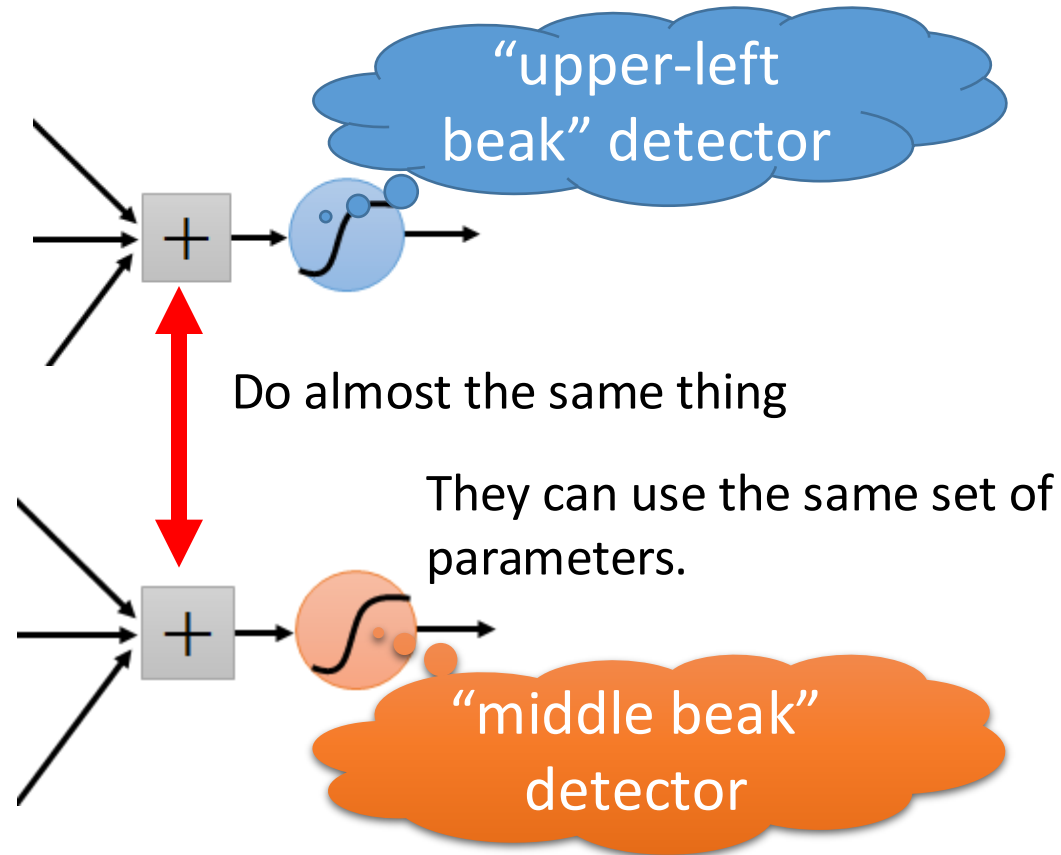
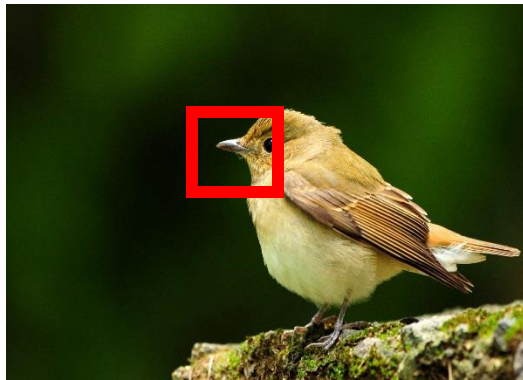
A neuron does not have to see the whole image to discover the pattern.

Connecting to small region with less parameters



Why CNN for image learning (2/3)

- The same patterns appear in different regions.



Why CNN for image learning (3/3)

- Subsampling the pixels will not change the object
 - Pictures now come with megapixel which means millions of pixels



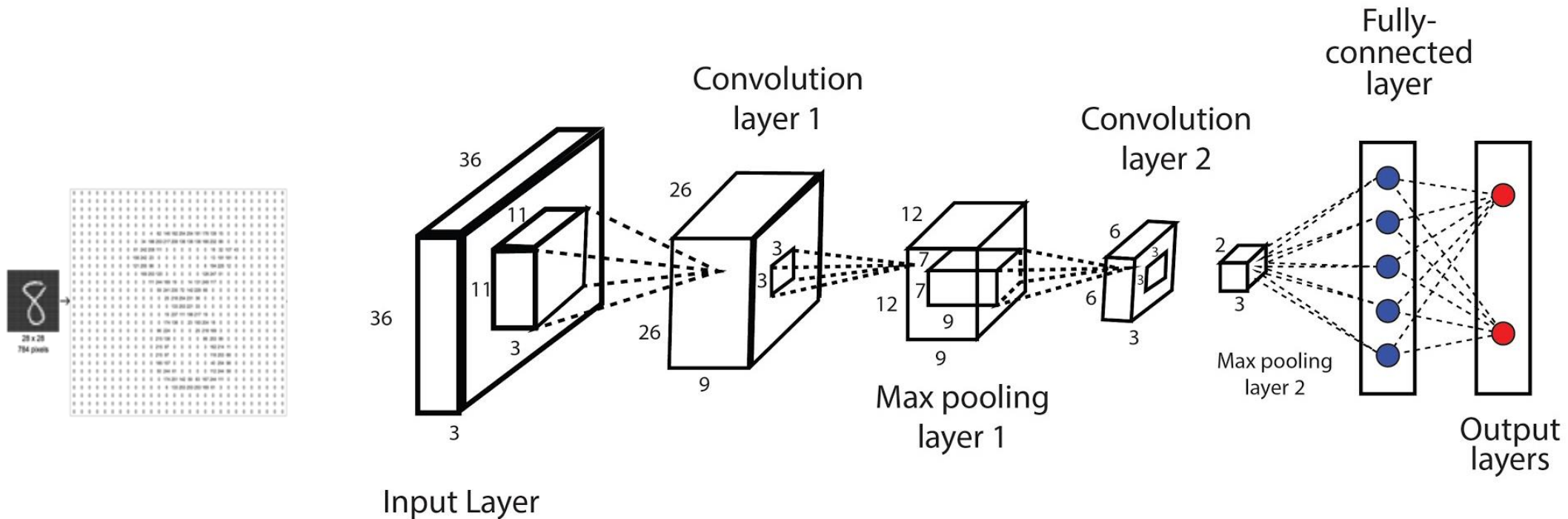
We can subsample the pixels to make image smaller

➡ Less parameters for the network to process the image

Why CNN for image learning

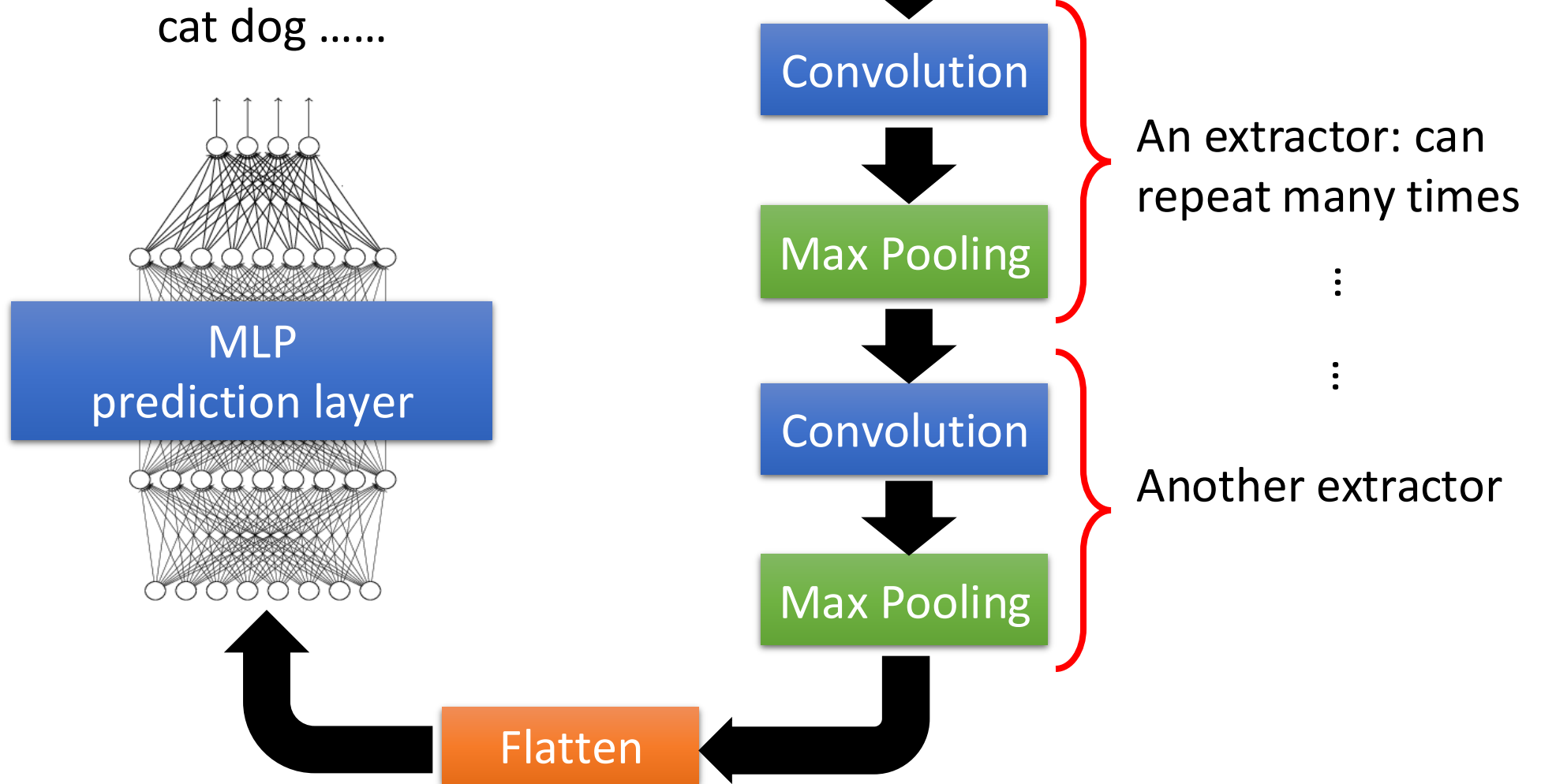
- Locality
 - Network should focus on **local regions**, without regard for the contents of the image in distant regions
 - These local representations can be **aggregated** to make predictions at the whole image level
- Translation invariance
 - Network should respond similarly to the same patch, regardless of where it appears in the image
- Subsampling
 - Less parameters for the network to process the image

A CNN structure

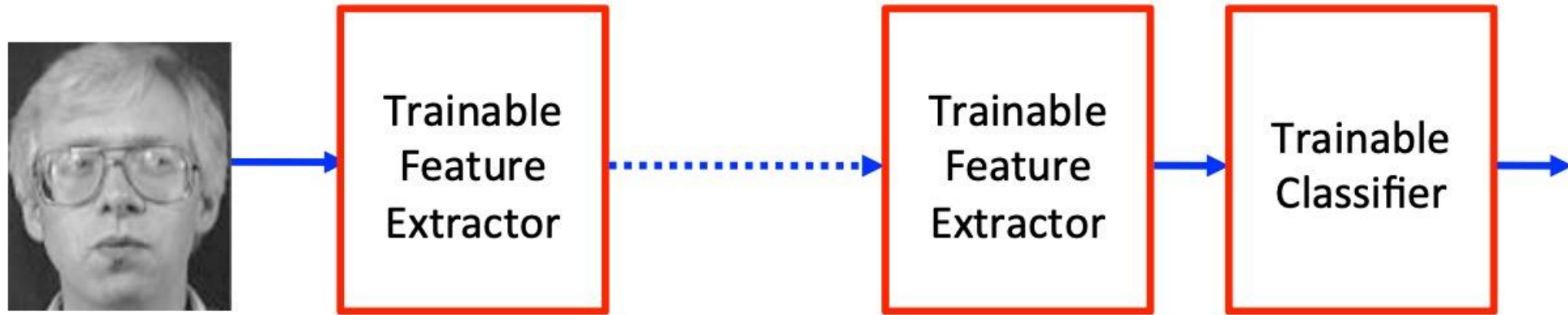


Three main types of layers: **Convolutional Layer**, **Pooling Layer**, and **Fully-Connected Layer**

The whole CNN



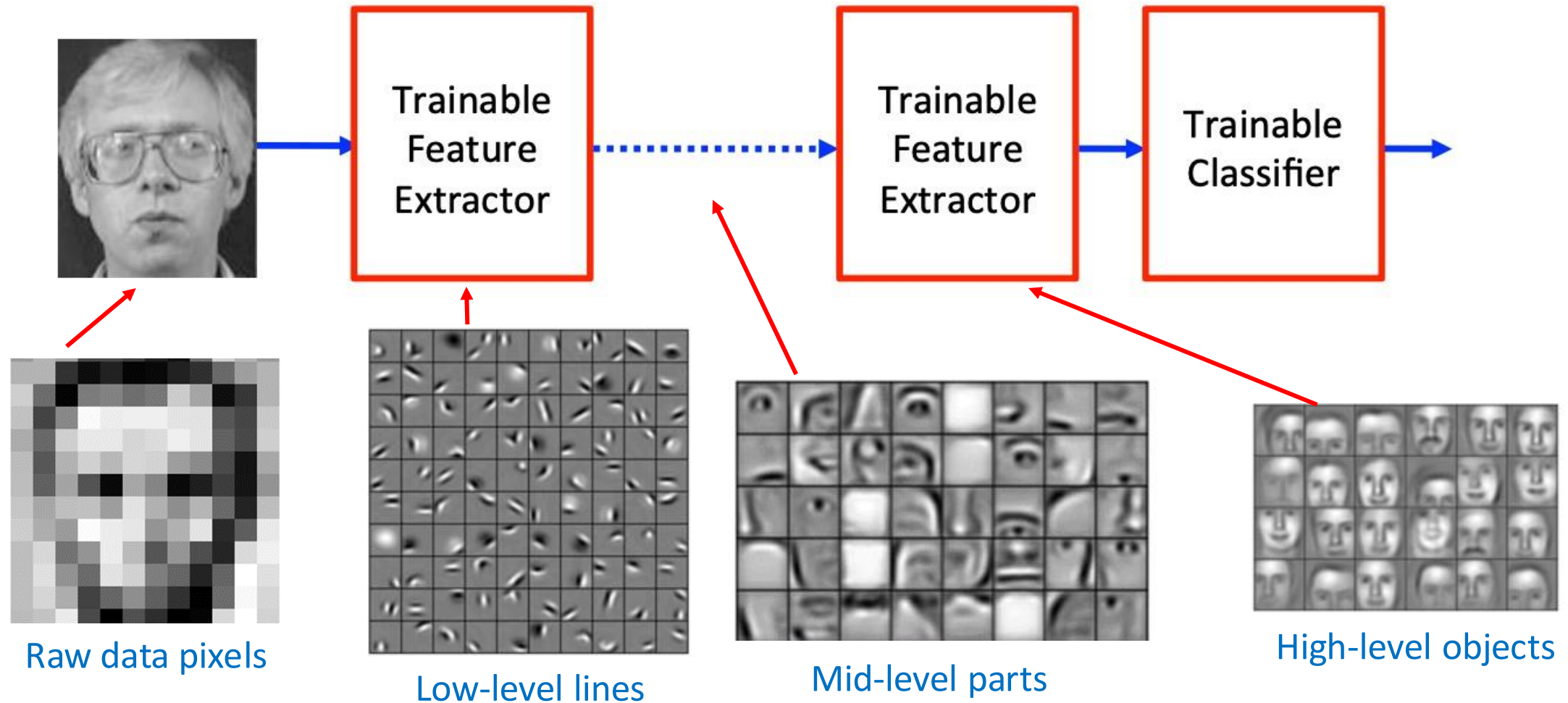
Hierarchical representation



Good image learning is hierarchical!

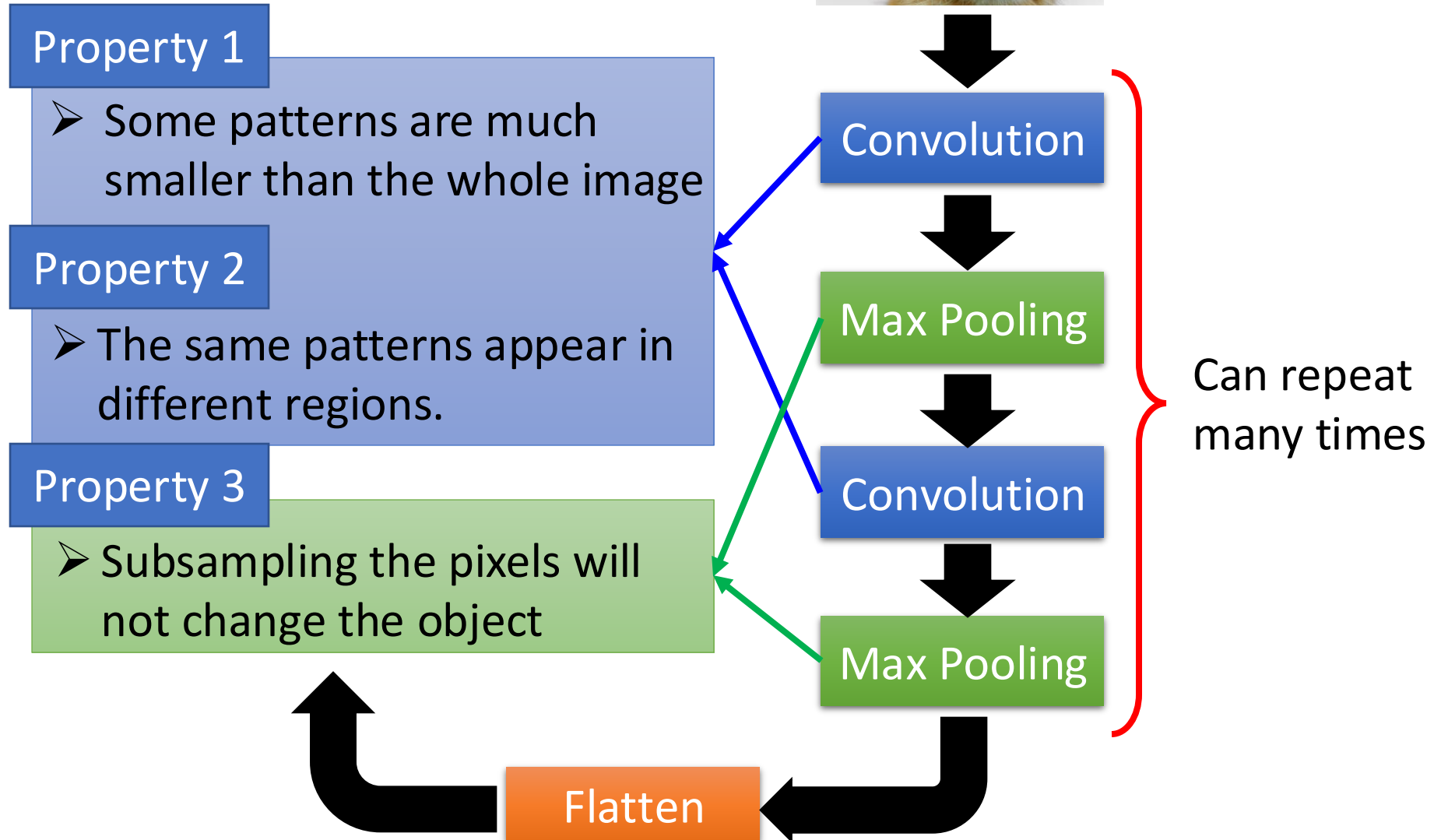
- To recognize a picture: part-to-whole hierarchy
 - Pixels ->Edges->Textons->Parts->Objects->Scenes
- Deep learning
 - Each extractor learns a hierarchy of representation at an internal layer
 - From low-level feature to mid-level representation to object identities

Hierarchical representation by convolution



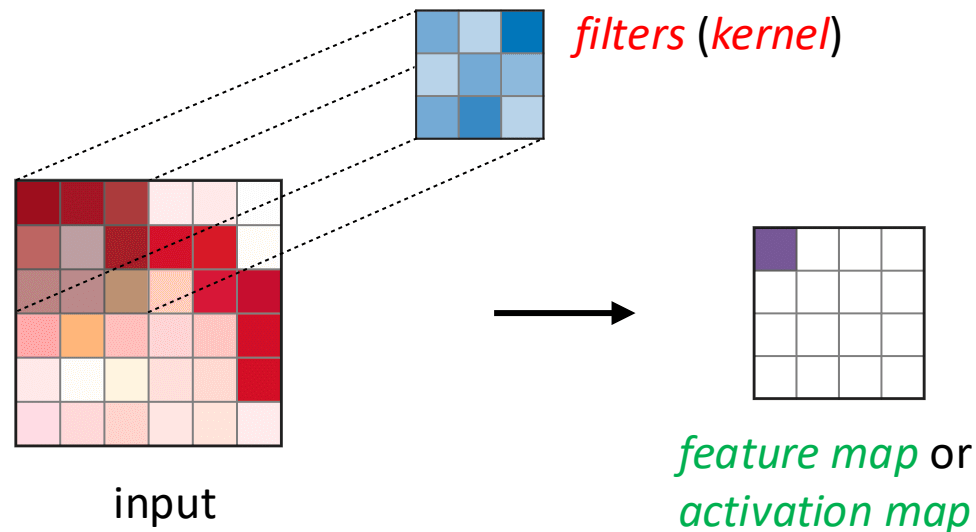
Good image learning is hierarchical – each level assemble the features from previous layer!

The whole CNN



Convolution layer

- The convolution layer uses *filters* that perform convolution operations
 - It scan the input I with respect to its dimensions.
- Its hyperparameters include the *filter size* F and *stride* S .
- The resulting output O is called *feature map* or *activation map*.



Input				Kernel			Output	
0	1	2	*	0	1	=	19	25
3	4	5		2	3		37	43
6	7	8						

CNN – convolution

Those are the network parameters to be learned.

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1
Matrix

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2
Matrix

⋮

Property 1

Each filter detects a small pattern (3 x 3).

CNN – convolution

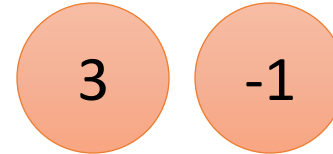
stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1



CNN – convolution

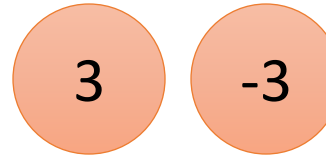
If stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1



We set stride=1 below

CNN – convolution

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

Property 2

CNN – Convolution

stride=1

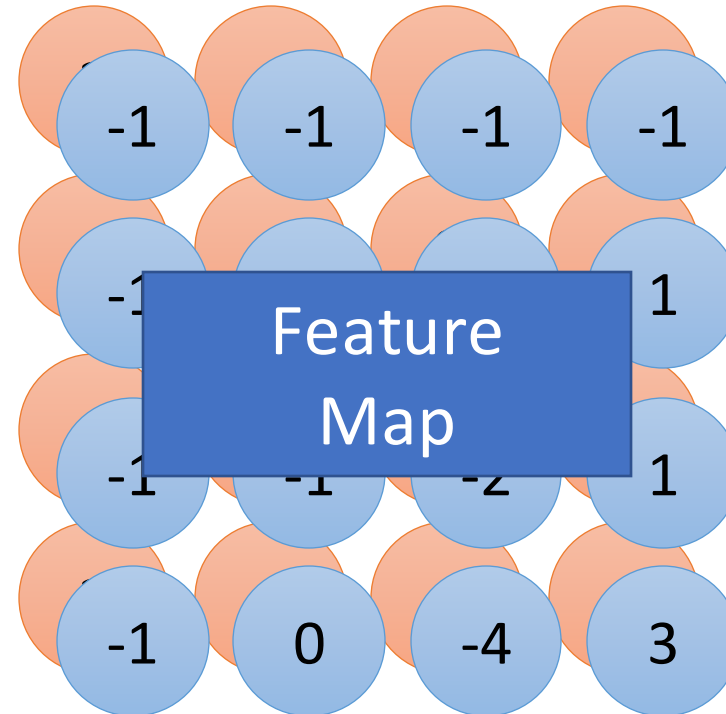
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

Do the same process for every filter



4 x 4 image

Padding and stride

- **Padding**: add extra pixels of filler around the boundary of our input image, thus increasing the effective size of the image

Input Kernel Output

0	0	0	0	0
0	0	1	2	0
0	3	4	5	0
0	6	7	8	0
0	0	0	0	0

 *

0	1
2	3

 =

0	3	8	4
9	19	25	10
21	37	43	16
6	7	8	0

- **Stride**: the number of rows and columns traversed per slide as the *stride*

Input Kernel Output

0	0	0	0	0
0	0	1	2	0
0	3	4	5	0
0	6	7	8	0
0	0	0	0	0

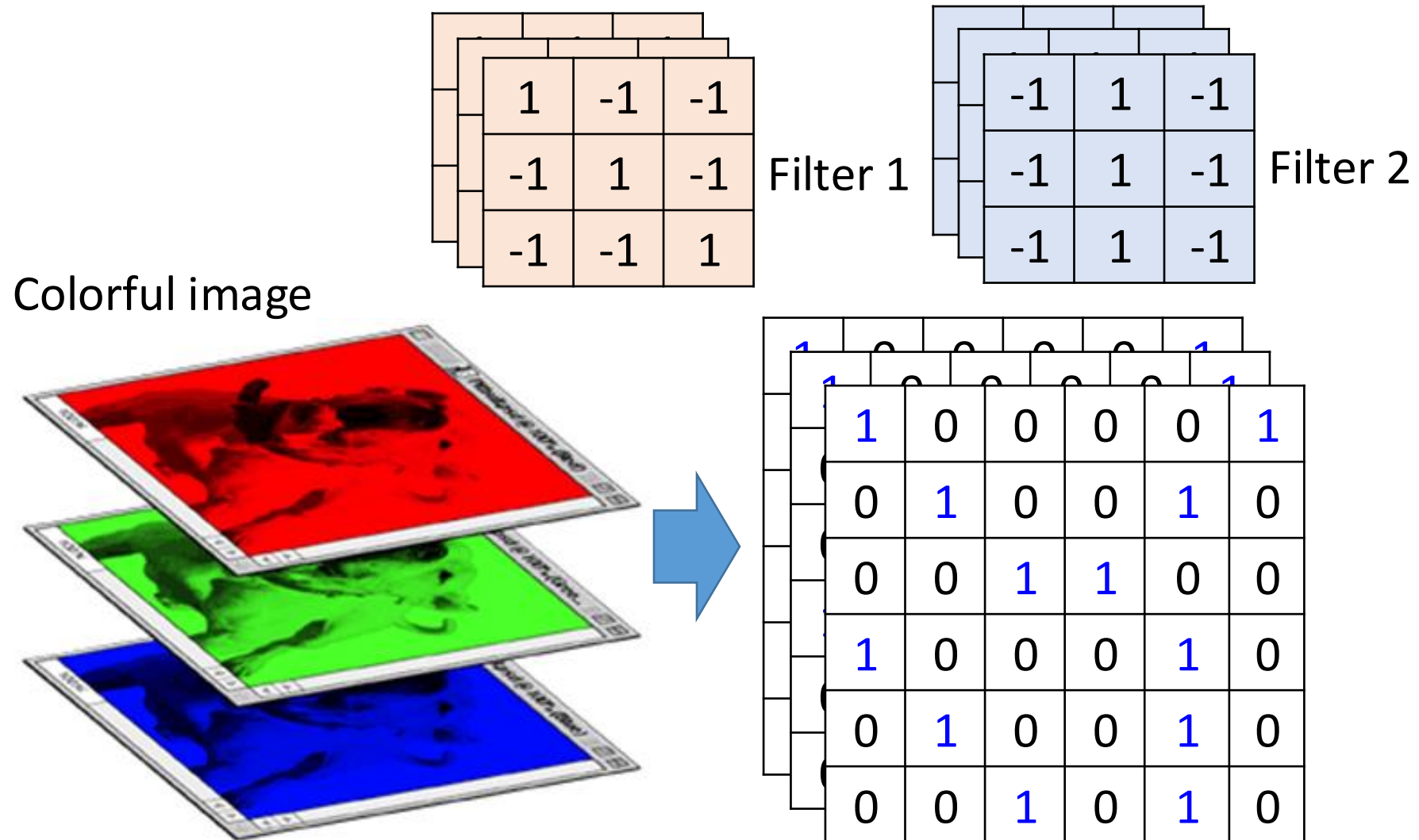
 *

0	1
2	3

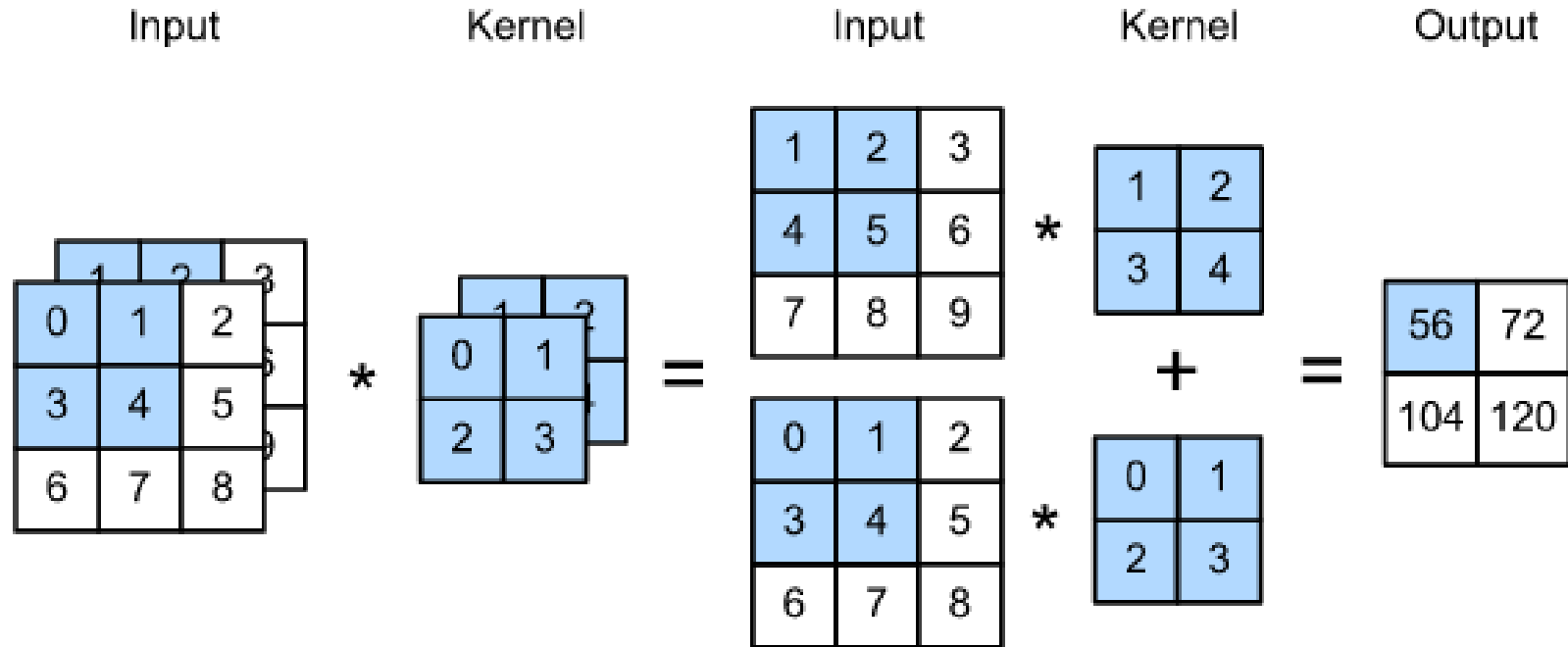
 =

0	8
6	8

CNN – colorful image

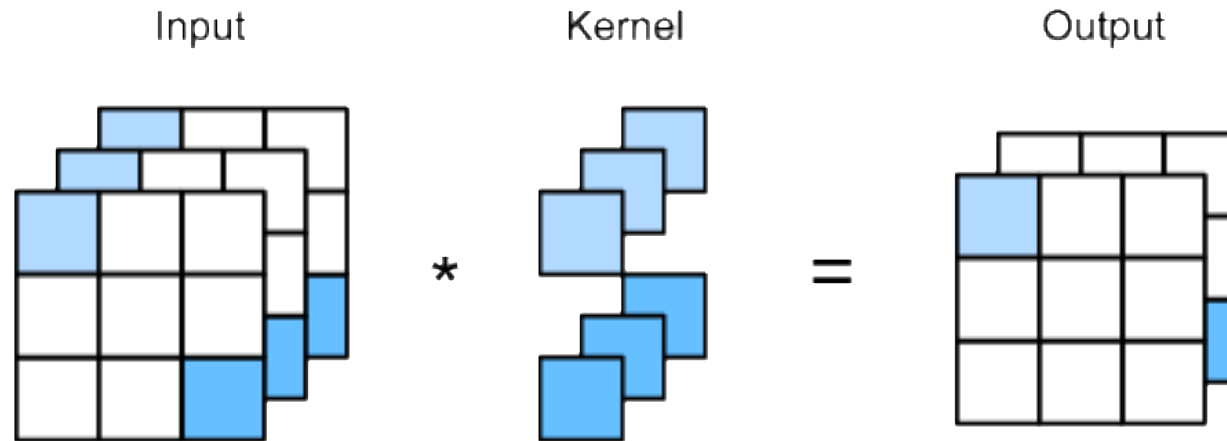


Multiple input channels



When the input data contain multiple channels, we need to construct a convolution kernel with the same number of **input channels** as the input data

Multiple output channels



When the output contain multiple channels, we need to construct multiple convolution kernels with the same number of channels of the outputs

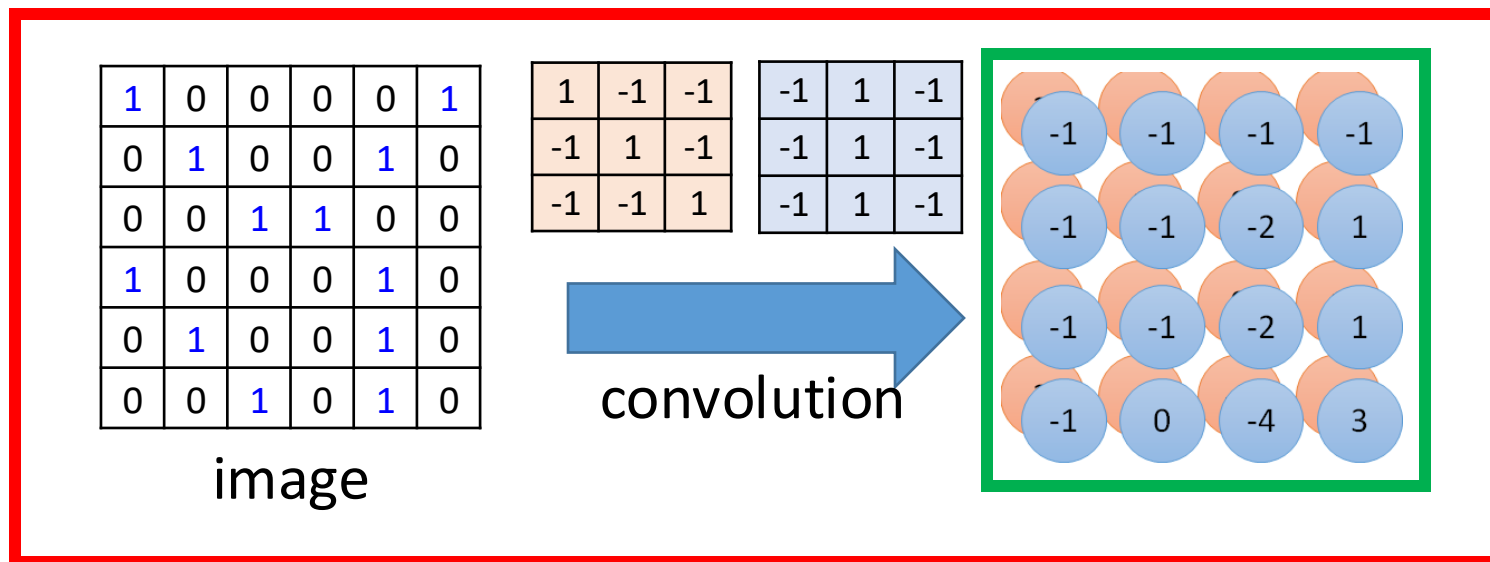
Conv layer in PyTorch

```
torch.nn.Conv2d(in_channels: int, out_channels: int, kernel_size: Union[T, Tuple[T, T]], stride: Union[T, Tuple[T, T]] = 1, padding: Union[T, Tuple[T, T]] = 0, dilation: Union[T, Tuple[T, T]] = 1, groups: int = 1, bias: bool = True, padding_mode: str = 'zeros')
```

```
nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5, stride=1, padding=2)
```

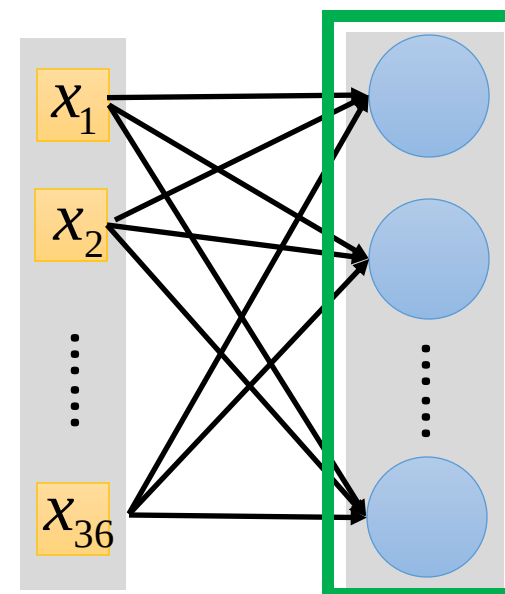
- `in_channels` ([int](#)) – Number of channels in the input image
- `out_channels` ([int](#)) – Number of channels produced by the convolution
- `kernel_size` ([int](#) or [tuple](#)) – Size of the convolving kernel
- `stride` ([int](#) or [tuple](#), *optional*) – Stride of the convolution. Default: 1
- `padding` ([int](#) or [tuple](#), *optional*) – Zero-padding added to both sides of the input. Default: 0
- `padding_mode` (*string, optional*) – 'zeros', 'reflect', 'replicate' or 'circular'. Default: 'zeros'
- `dilation` ([int](#) or [tuple](#), *optional*) – Spacing between kernel elements. Default: 1
- `groups` ([int](#), *optional*) – Number of blocked connections from input channels to output channels. Default: 1
- `bias` ([bool](#), *optional*) – If True, adds a learnable bias to the output. Default: True

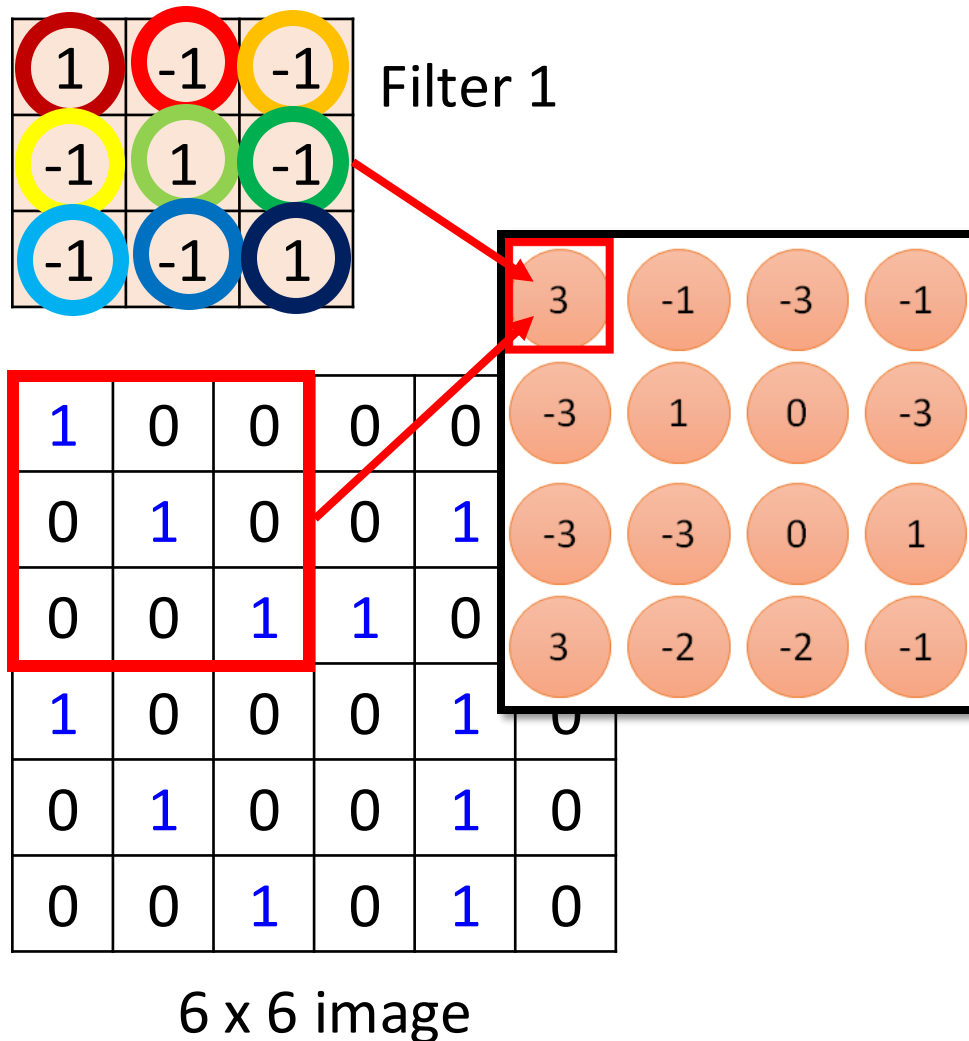
Convolution v.s. Fully Connected



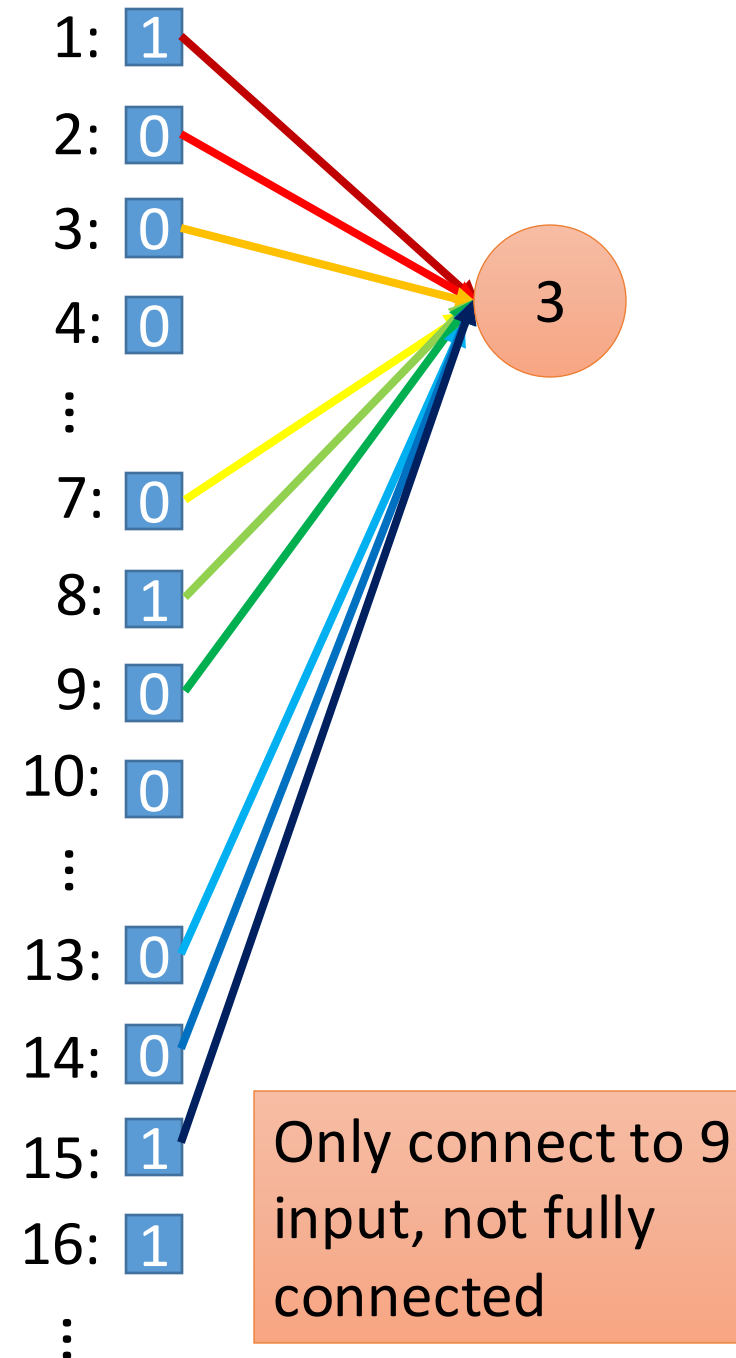
Fully-
connected

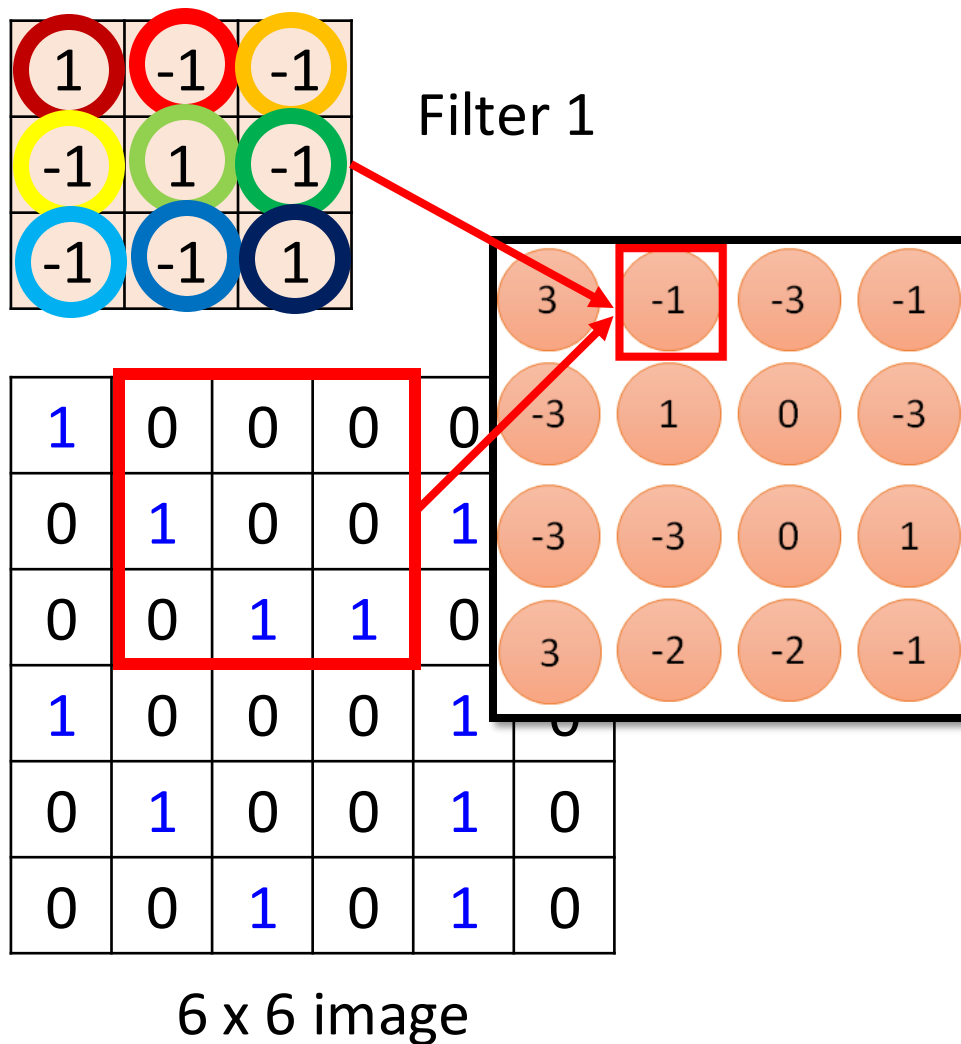
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0





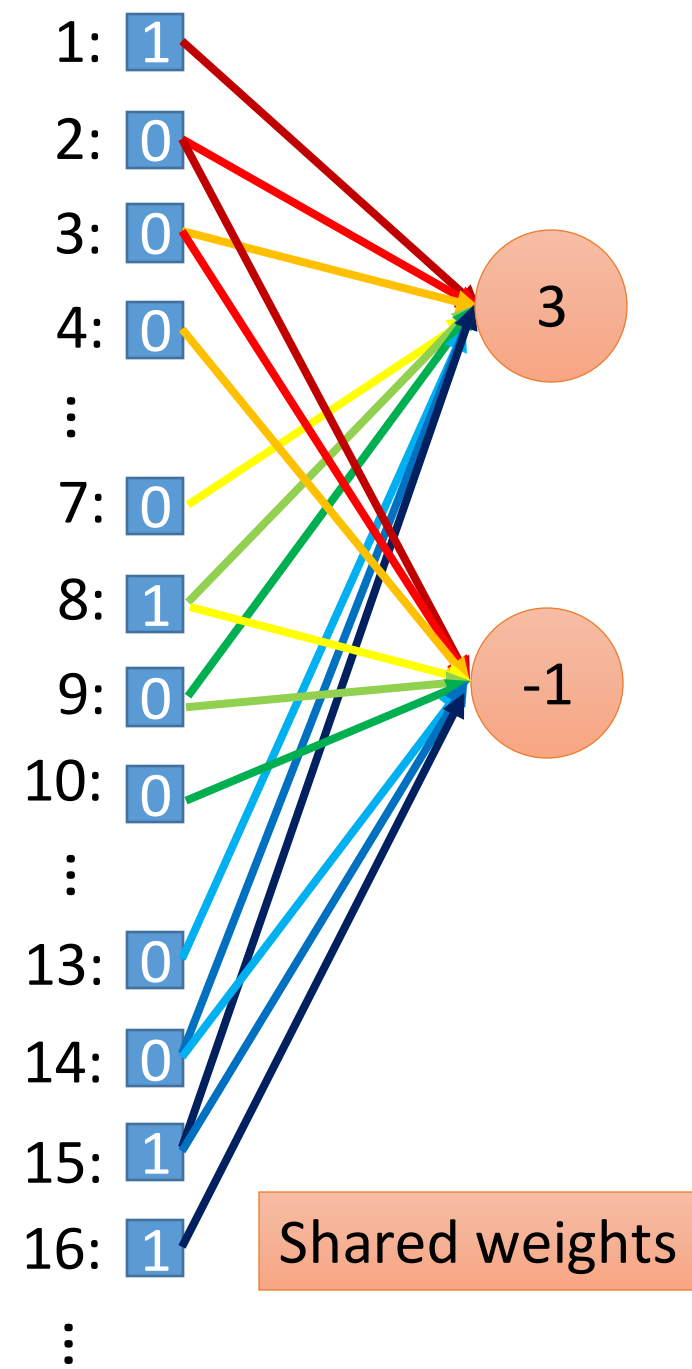
Less parameters!





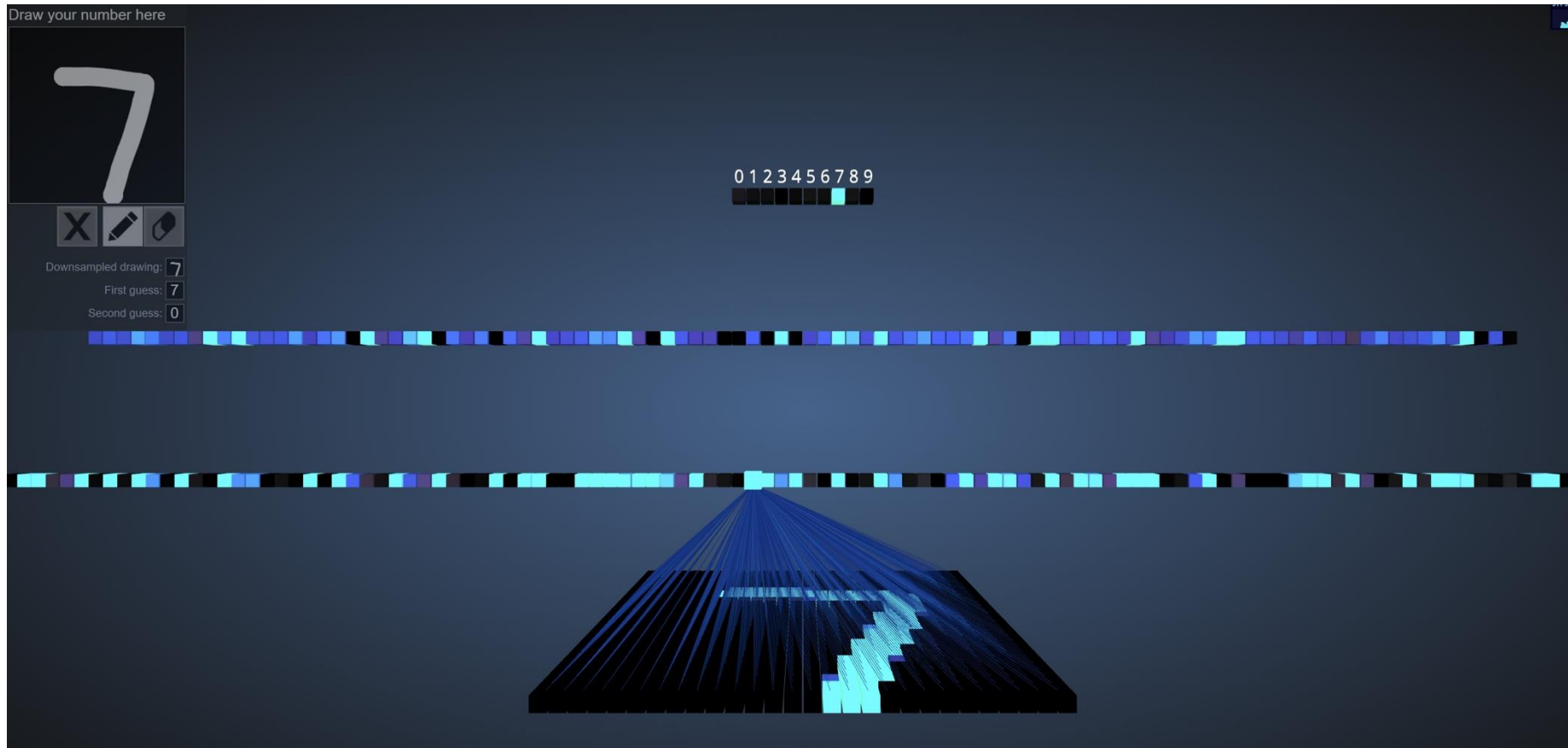
Less parameters!

Even less parameters!

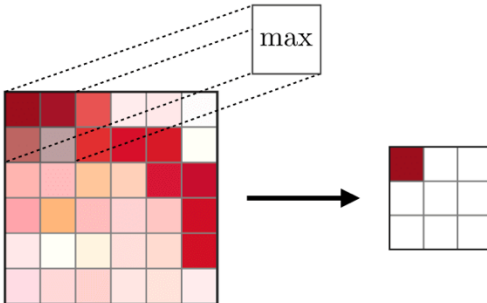
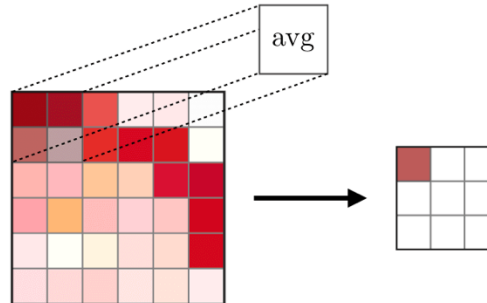


Convolution v.s. Fully Connected

- Visualizer: https://adamharley.com/nn_vis/



Pooling layer

Type	Max pooling	Average pooling
Purpose	Each pooling operation selects the maximum value of the current view	Each pooling operation averages the values of the current view
Illustration		
Comments	• Preserves detected features • Most commonly used	• Downsamples feature map • Used in LeNet

The pooling layer is a downsampling operation, typically applied after a convolution layer !

CNN – max pooling

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

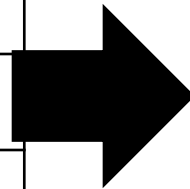
3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

CNN – max pooling

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

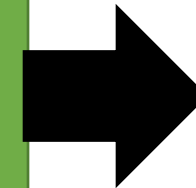
6 x 6 image



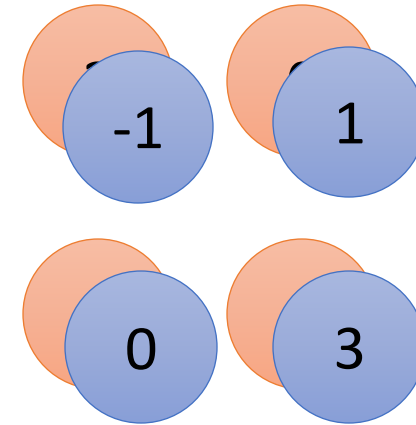
Conv



Max
Pooling



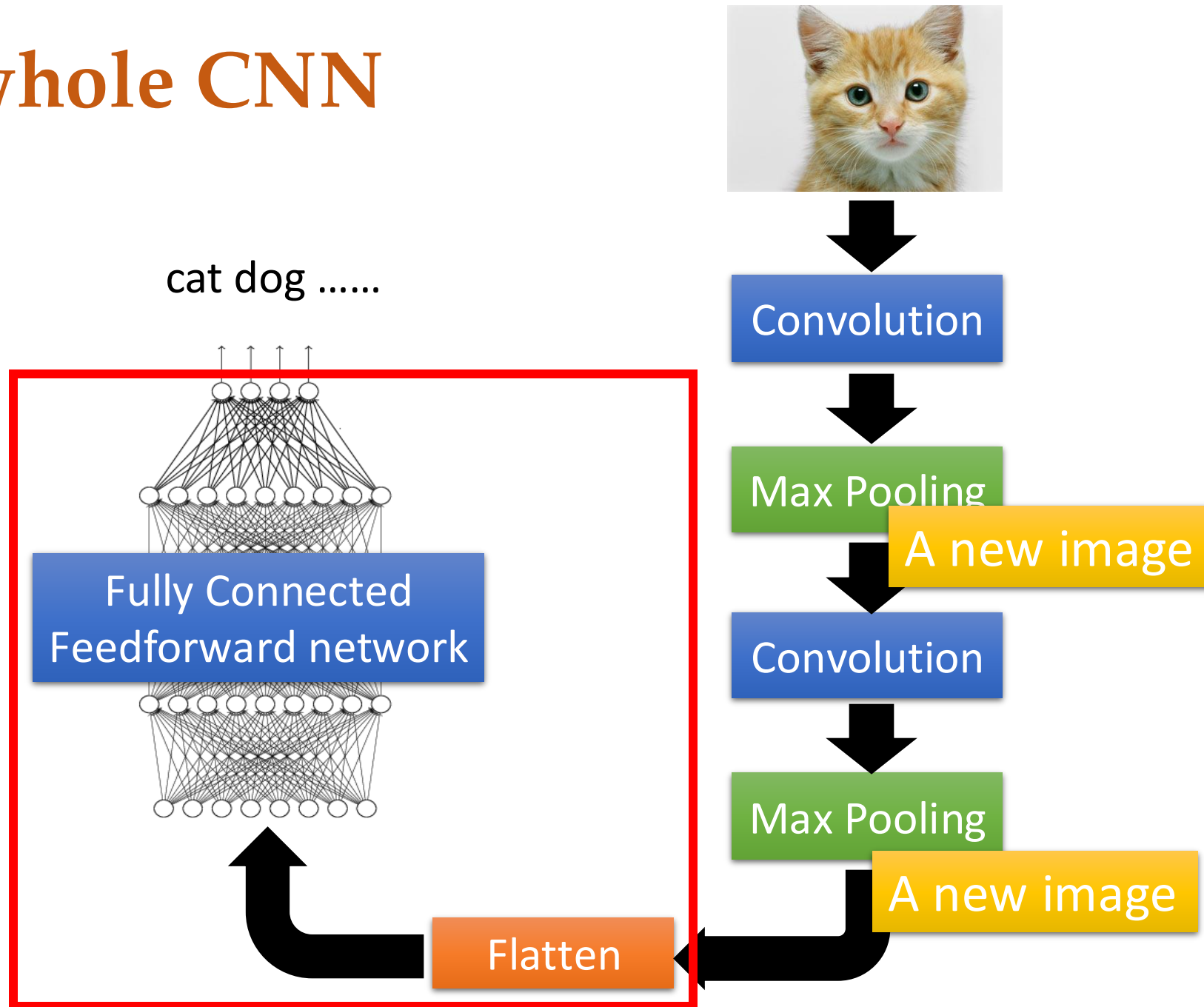
New image
but smaller



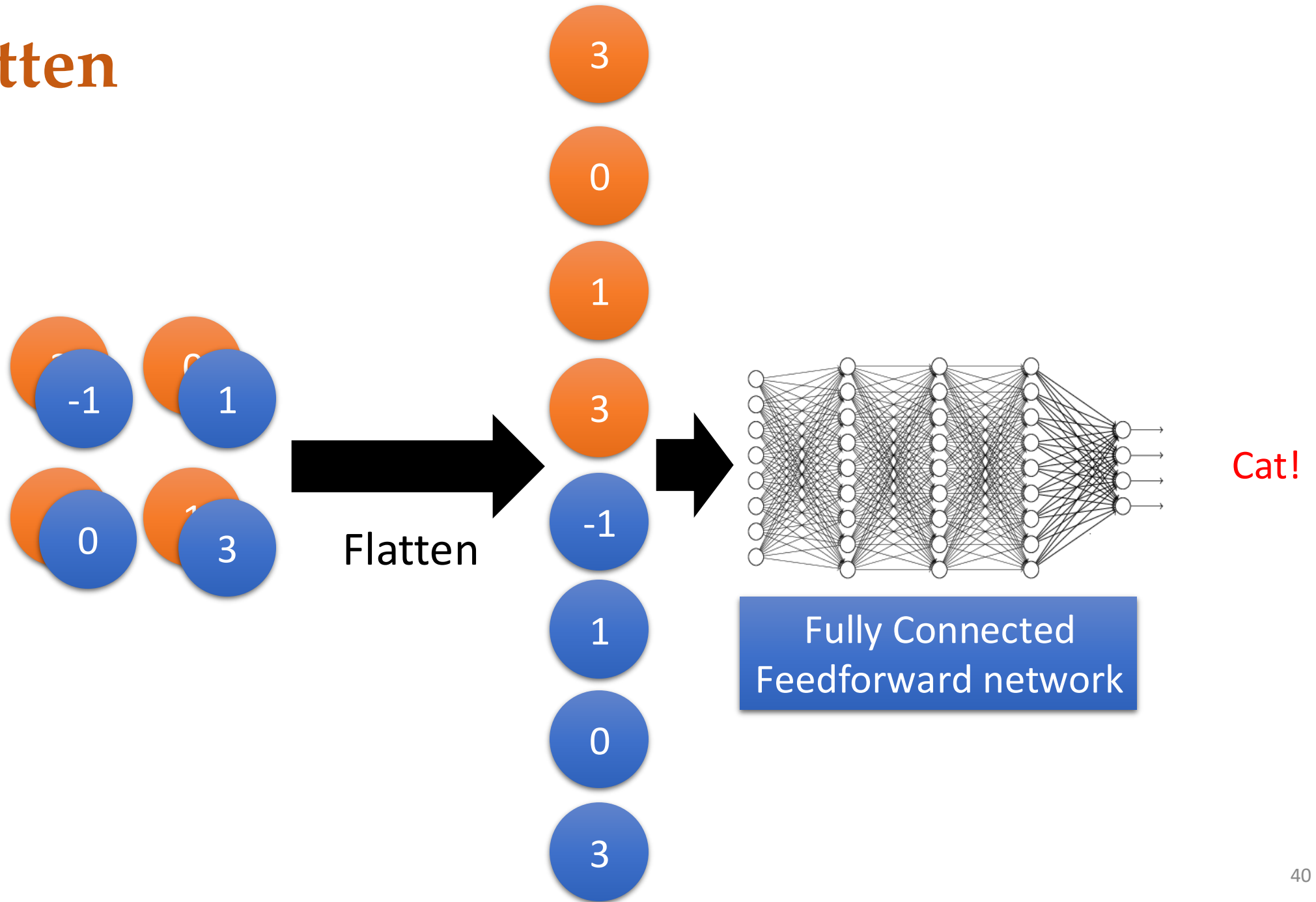
2 x 2 image

Each filter
is a channel

The whole CNN

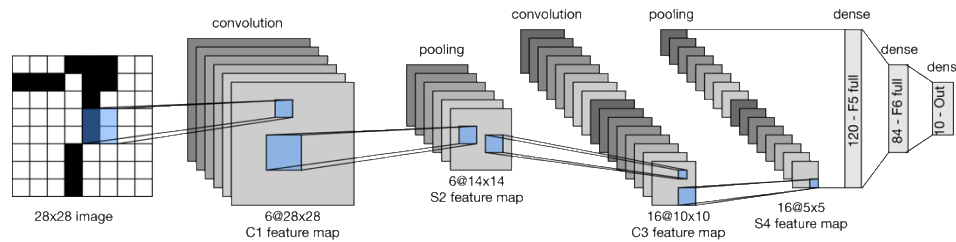


Flatten



Example: LeNet

LeCun, Yann, et al. "Gradient-based learning applied to document recognition."



Layer	Output Size	Weight Size
Input	1 x 28 x 28	
Conv (Cout=6, K=5, P=2, S=1)	6 x 28 x 28	6 x 1 x 5 x 5
ReLU	6 x 28 x 28	
MaxPool(K=2, S=2)	6 x 14 x 14	
Conv (Cout=16, K=5, S=1)	16 x 10 x 10	16 x 6 x 5 x 5
ReLU	16 x 10 x 10	
MaxPool(K=2, S=2)	16 x 5 x 5	
Flatten	16*5*5	
Linear (400 -> 120)	120	400 x 120
ReLU	120	
Linear (120 -> 84)	84	120 x 84
ReLU	120	
Linear (84 -> 10)	10	84 x 10

net = nn.Sequential(

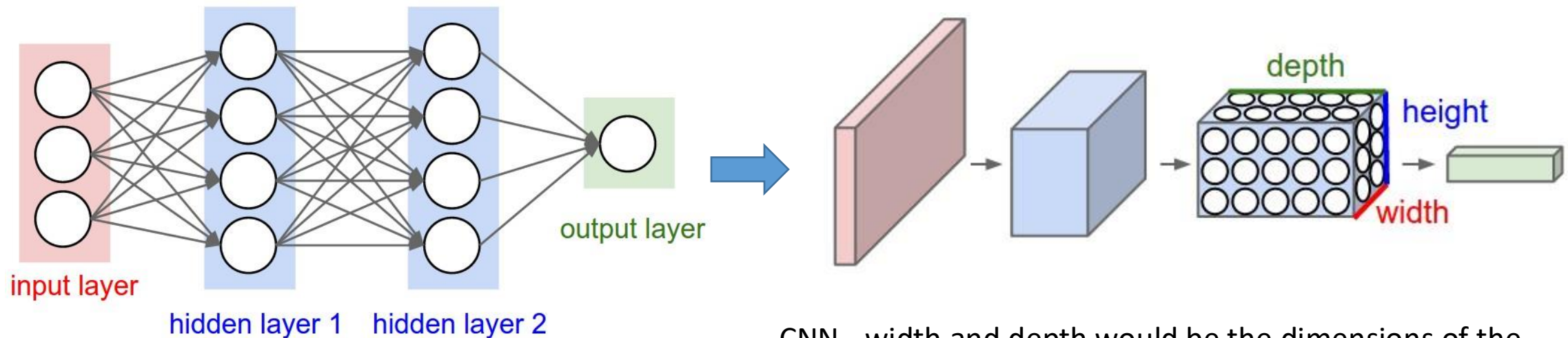
```
nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5, stride=1,
padding=2),
nn.relu(),
nn.MaxPool2d(kernel_size=2, stride=2),
```

```
nn.Conv2d(in_channels=6, out_channels= 16, kernel_size=5, stride=1,
padding=0),
nn.relu(),
nn.MaxPool2d(kernel_size=2, stride=2),
```

```
nn.Flatten(),
nn.Linear(16*5*5, 120),
nn.relu(),
nn.Linear(120, 84),
nn.relu(),
nn.Linear(84, 10))
```

Why CNN for image learning

- Comparing to record data where each data point is a vector
 - Image data is a matrix with **grid-like topology**
 - Image data is a tensor consisting of **multiple channels (RGB)**
 - Image data is a high-dimensional data require **computational efficiency**



Fully-connected layers

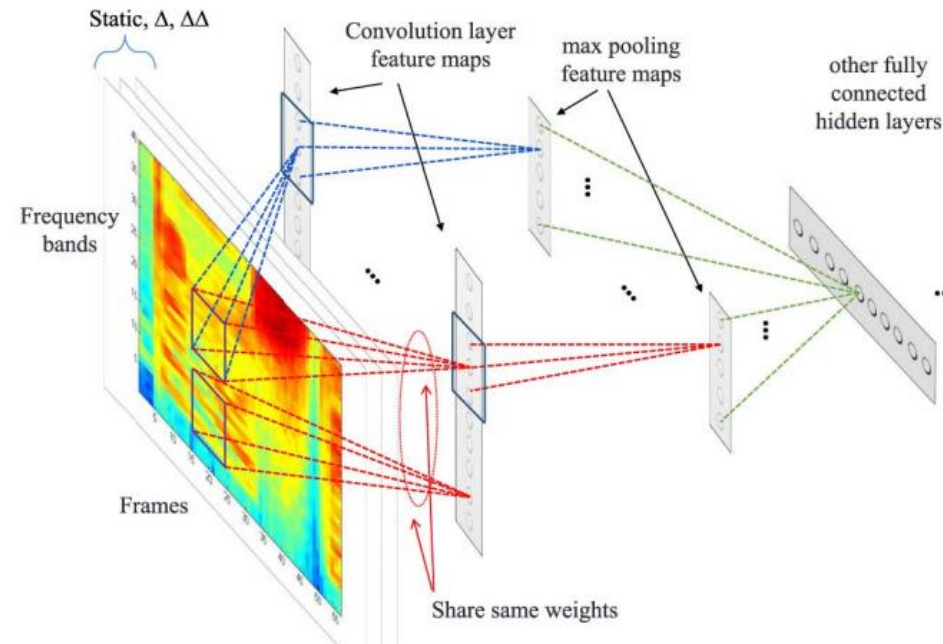
CNN - width and depth would be the dimensions of the image, and height would be 3 (Red, Green, Blue channels)

[Fei-Fei Li, Stanford CS 231](#)

More applications of CNN

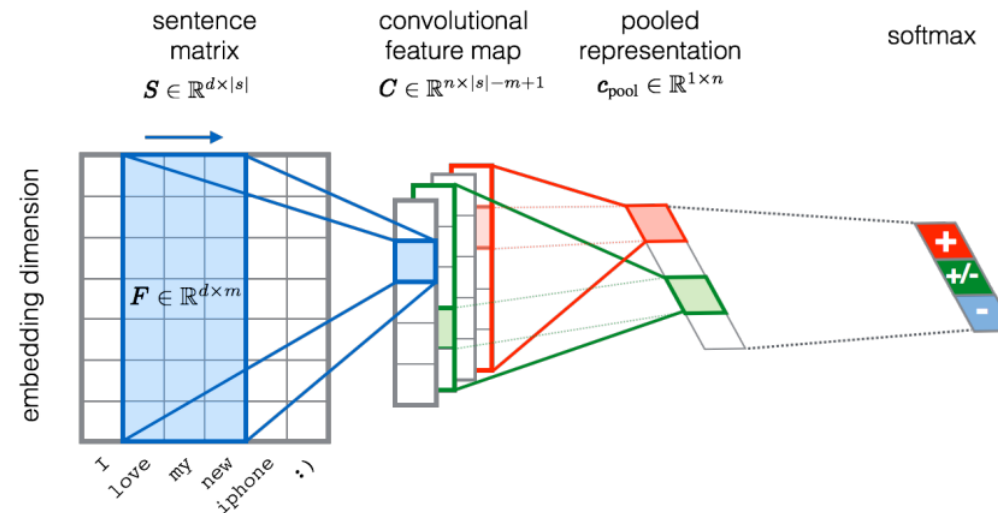
Speech

<https://dl.acm.org/doi/10.1109/TASLP.2014.2339736>



Natural Language Processing

<https://www.aclweb.org/anthology/S15-2079/>



Agenda

- Convolutional Neural Networks (CNN)
 - What is CNN
 - Convolutional Layer
 - Pooling Layer
- Recurrent Neural Networks (RNN)
 - What is RNN
 - Computation of RNN
 - Language Modeling
 - Long Short Term Memory (LSTM) Networks

Image captioning



A cat sitting on a suitcase on the floor



A cat is sitting on a tree branch



A dog is running in the grass with a frisbee



A white teddy bear sitting in the grass



Two people walking on the beach with surfboards



A tennis player in action on the court

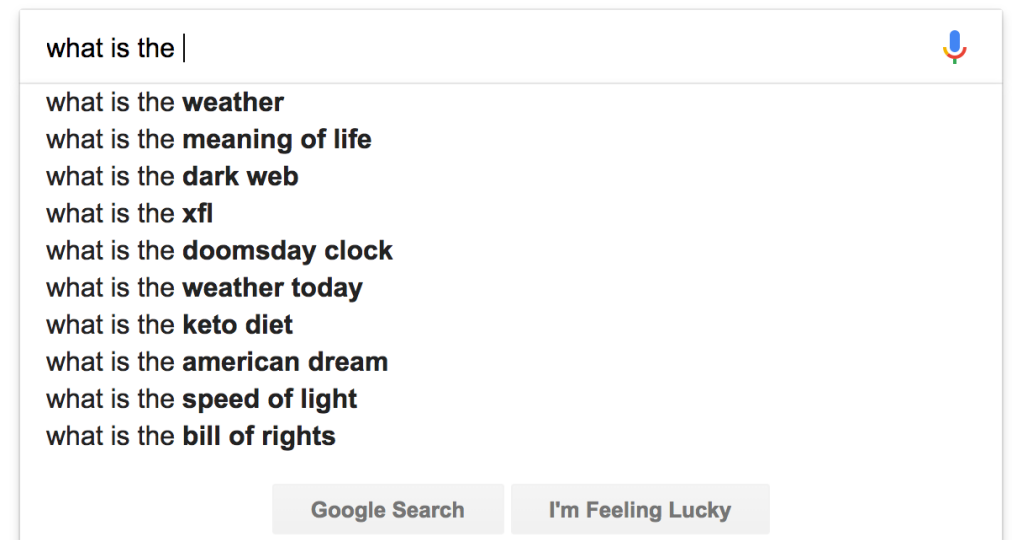
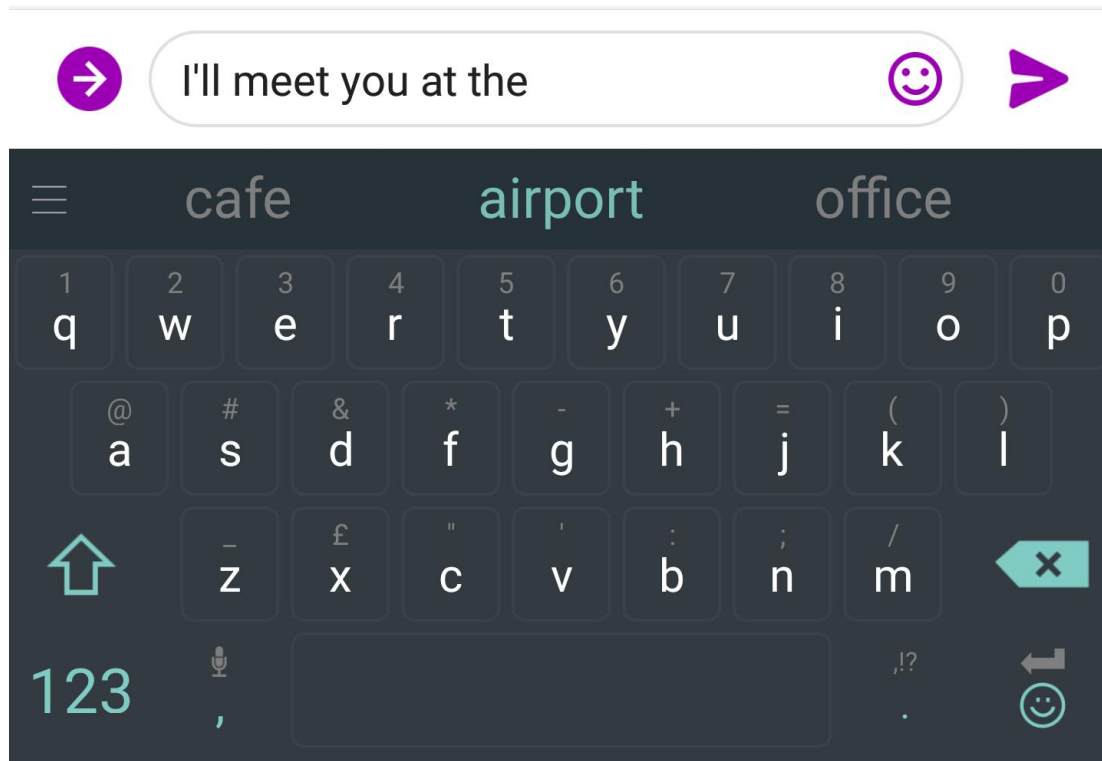


Two giraffes standing in a grassy field

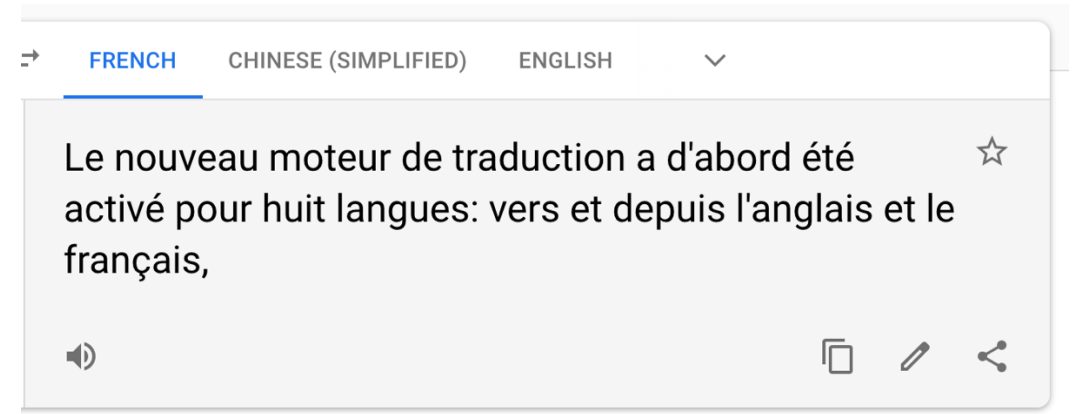
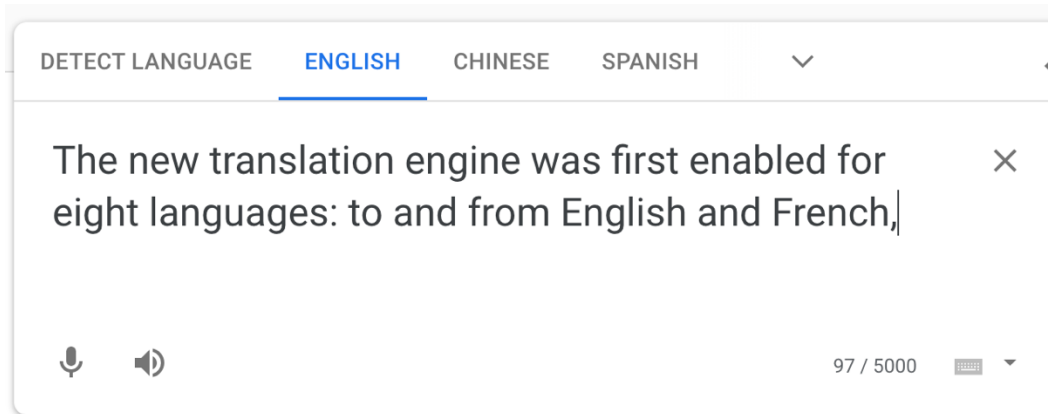


A man riding a dirt bike on a dirt track

Autocomplete prediction



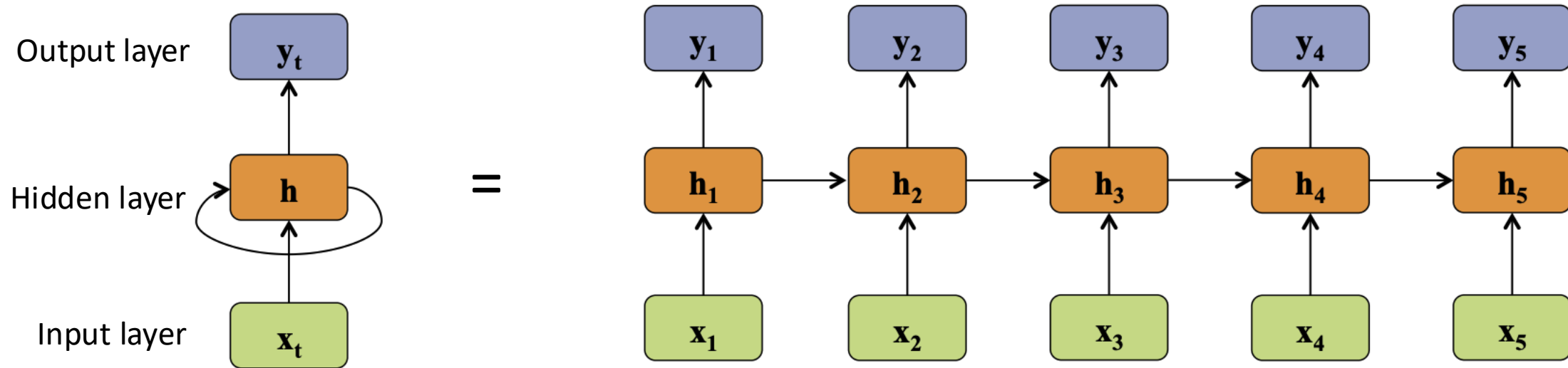
Machine translation



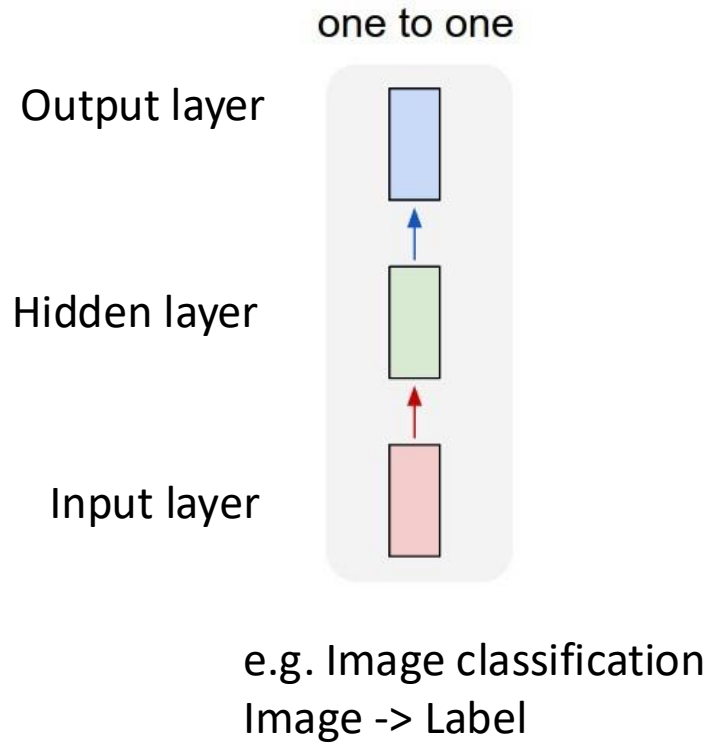
[Send feedback](#)

Recurrent Neural Networks (RNNs)

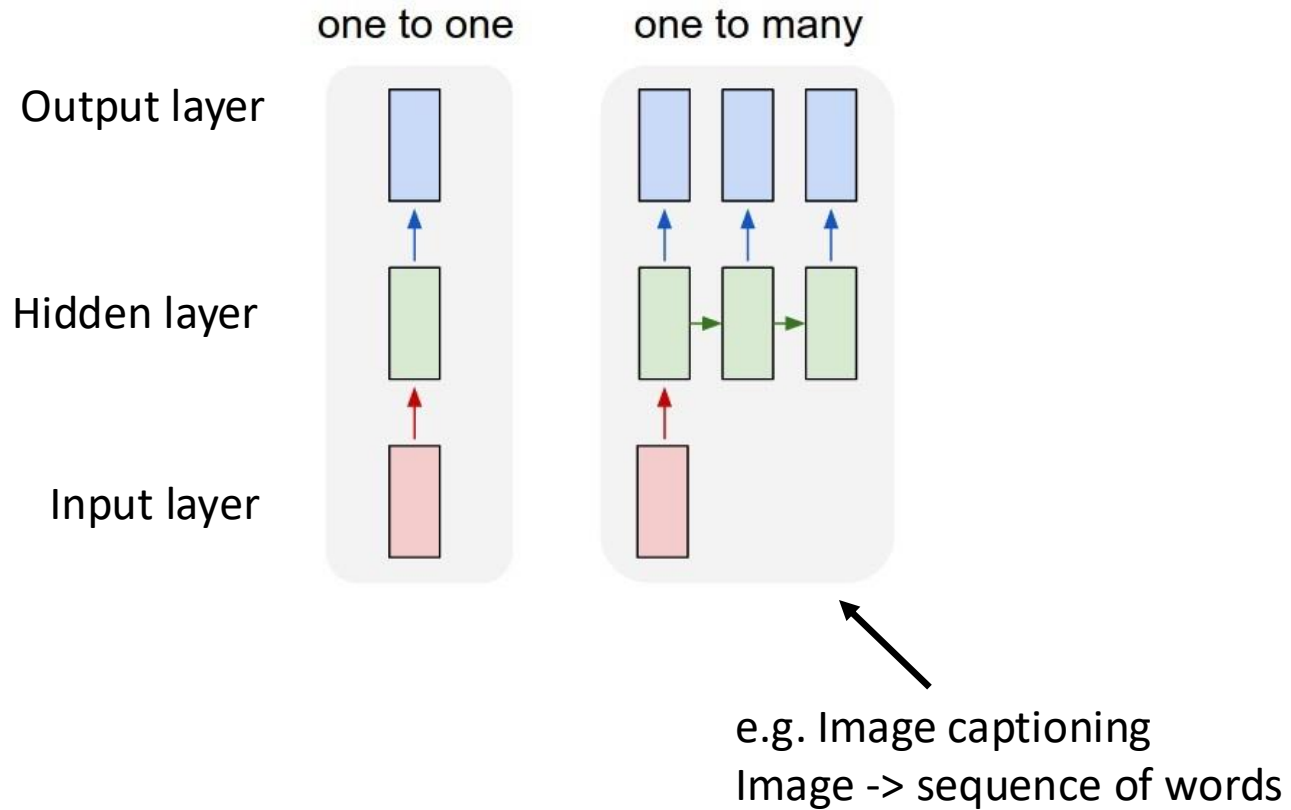
- RNNs are designed to handle **sequential information**
 - CNNs are designed to process image data
- RNNs introduce **state variables** to memorize past information, combine with the **current inputs**, to determine the current outputs



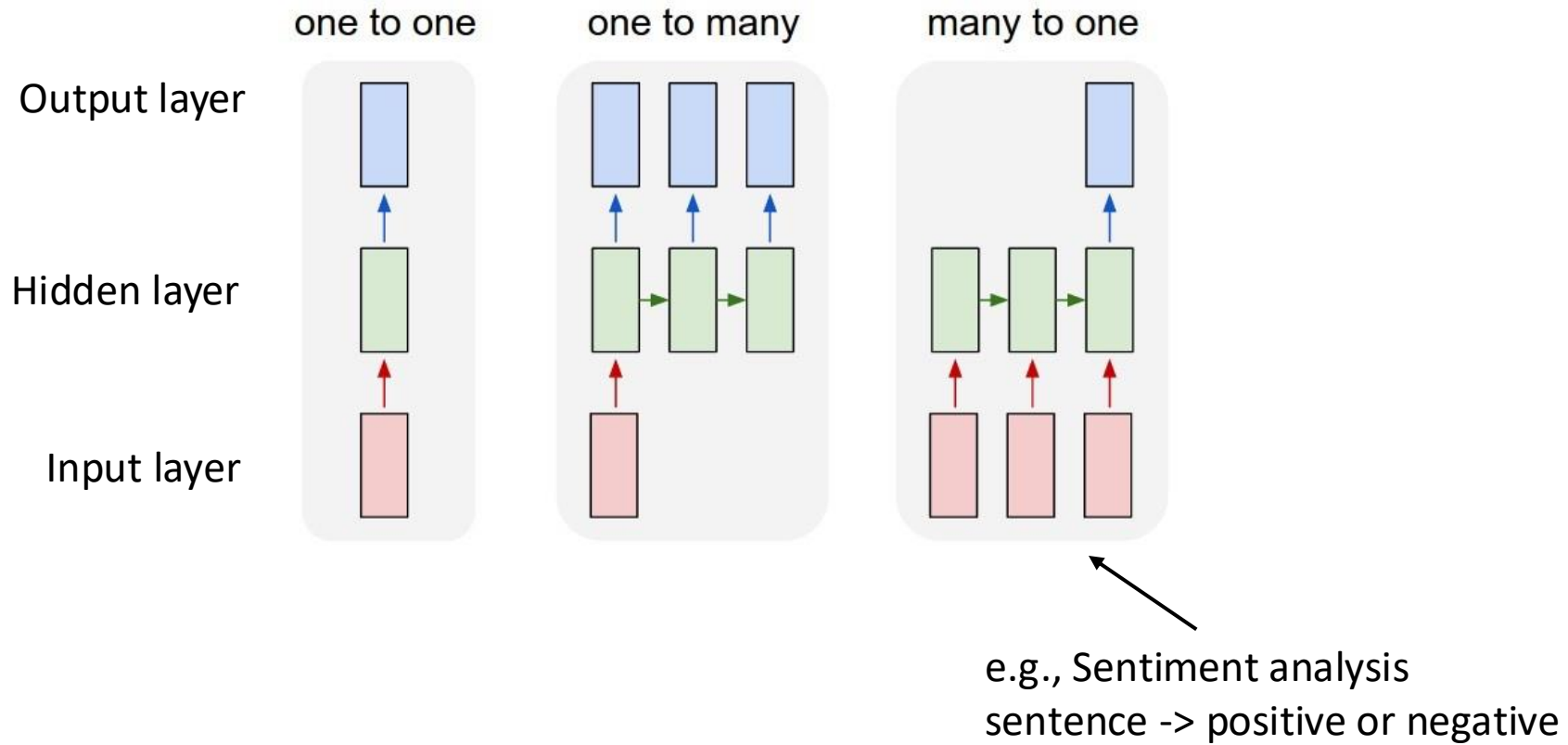
Ways to process sequences data



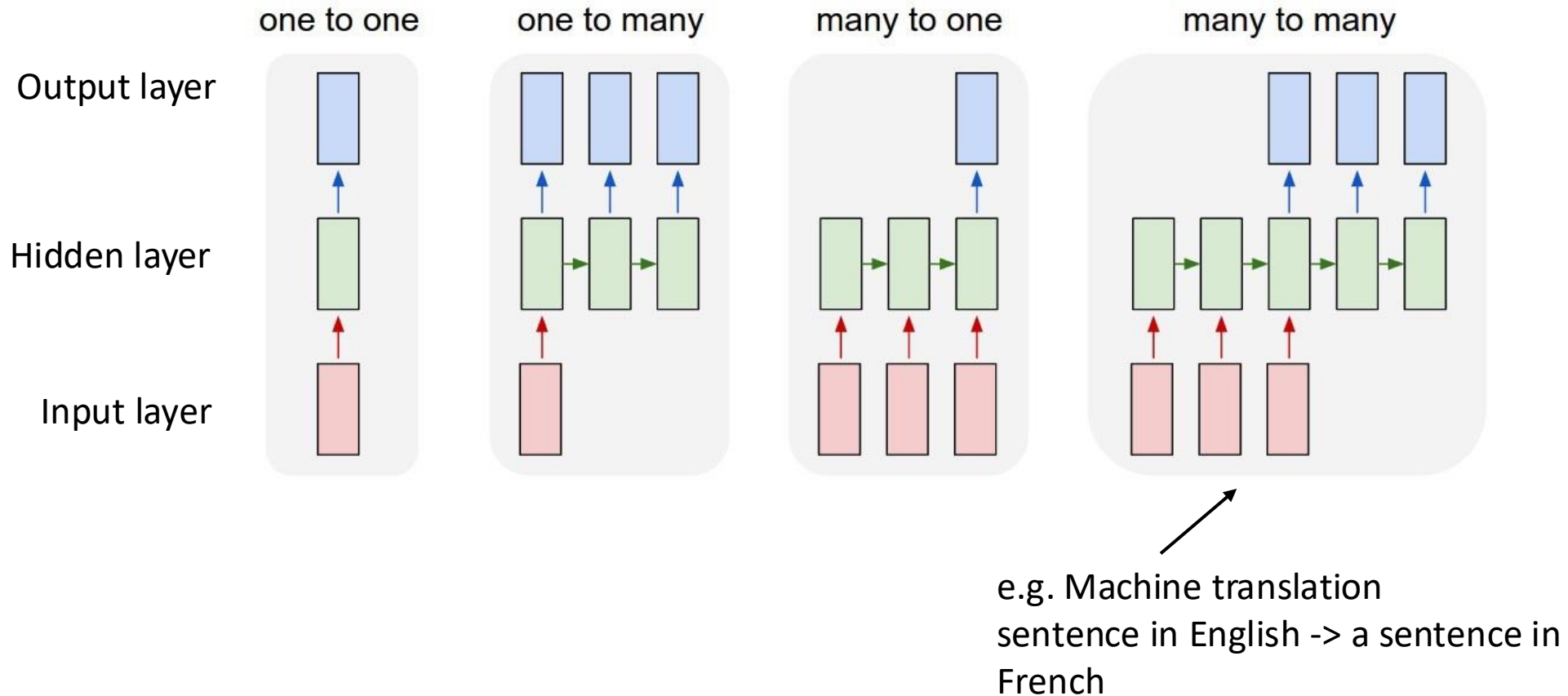
Ways to process sequences data



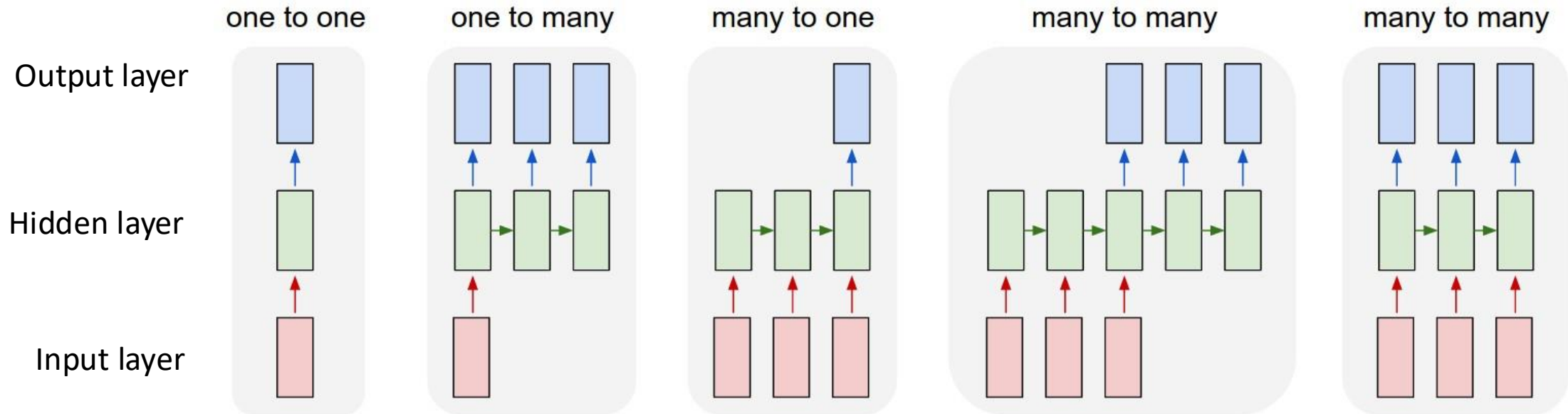
Ways to process sequences data



Ways to process sequences data



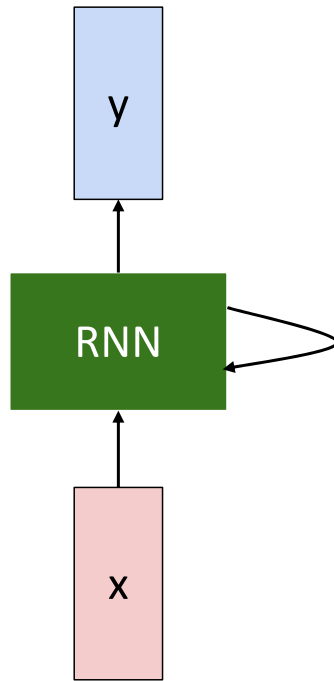
Ways to process sequences data



e.g., Per-frame video classification
Sequence of images -> Sequence of labels

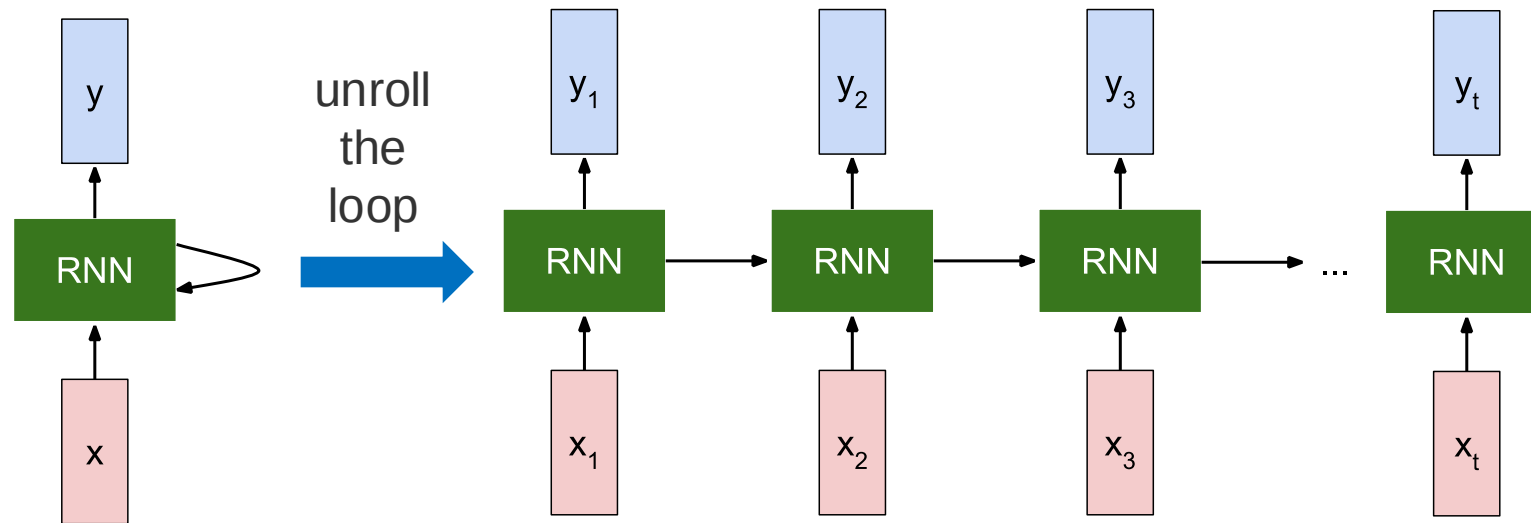
Same: they all maintain and update a “memory” during the sequence

How does RNN work



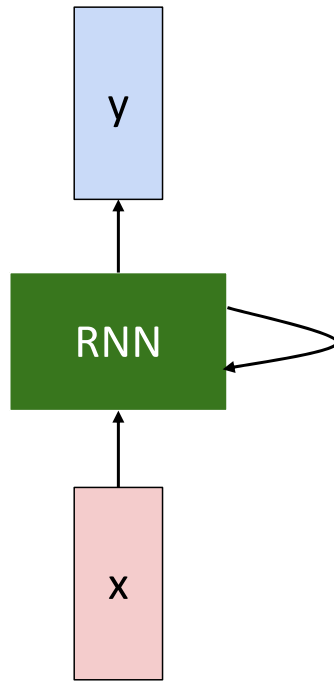
Key idea: RNNs have an “**internal state**” (hidden state) that is updated as a sequence is processed

Computation of RNN



A recurrent neural network can be thought of as multiple copies of the same network, each passing a message to a successor

Computation of RNN



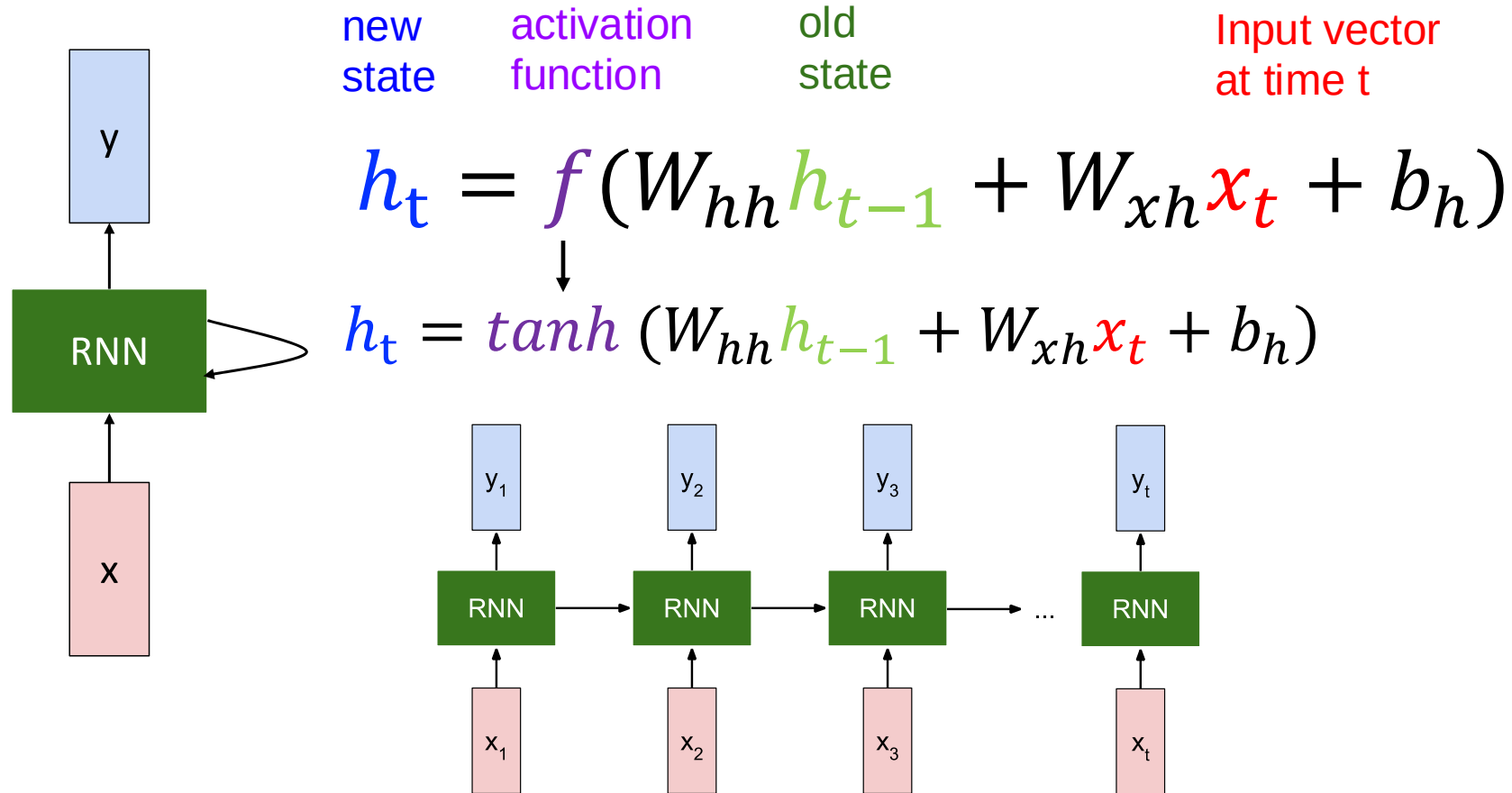
We can process a sequence of vectors x by applying a recurrence formula at every time step:

new state activation function old state Input vector at time t

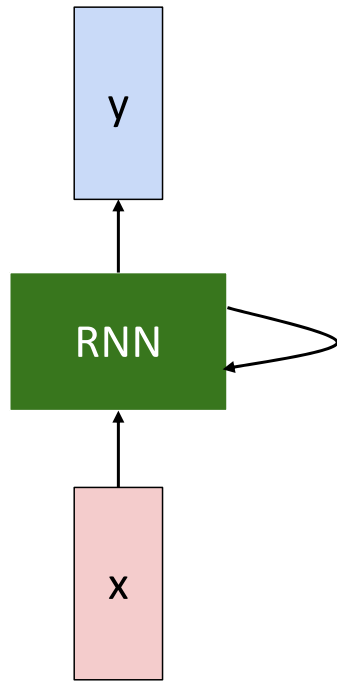
$$h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

Notice: the same function and the same set of parameters are used at every time step.

Computation of RNN



Recurrent Neural Networks



new
state

activation
function

old
state

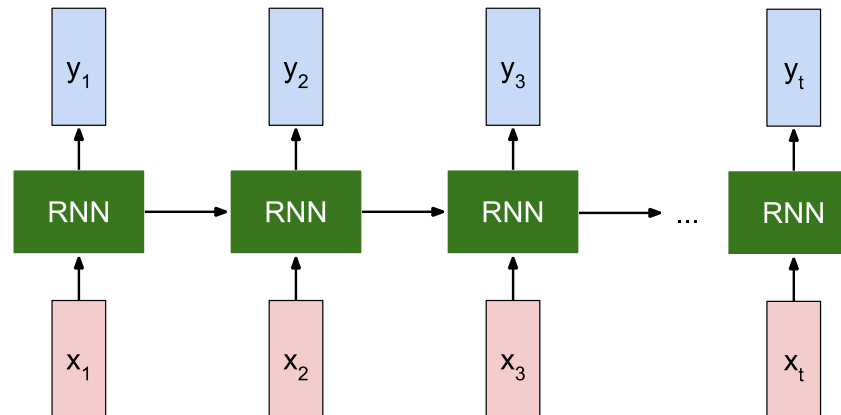
Input vector at
time t

$$h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

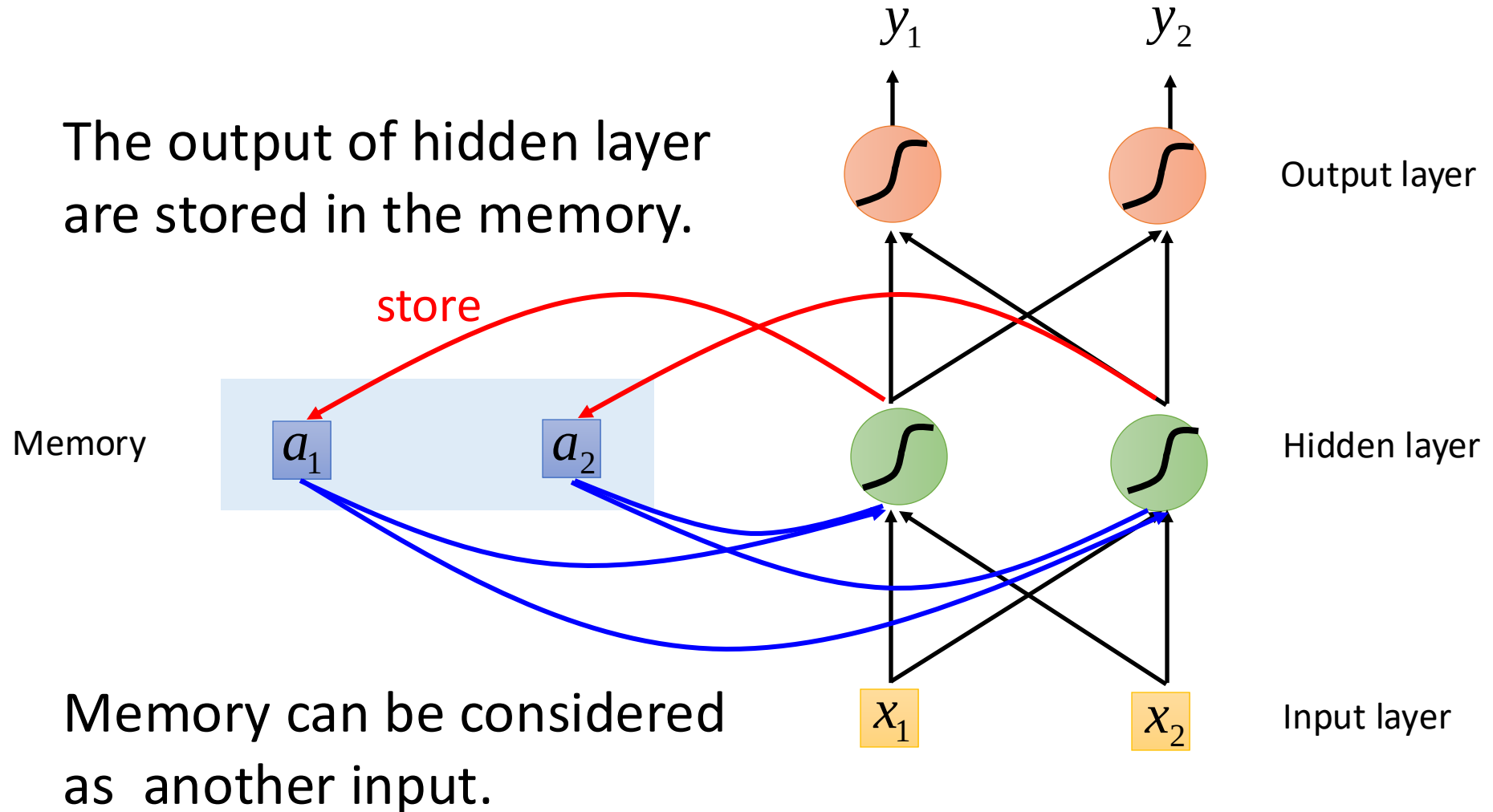
Hidden layer at
each time step

$$y_t = g(W_{hy}h_t + b_y)$$

Output layer at
each time step



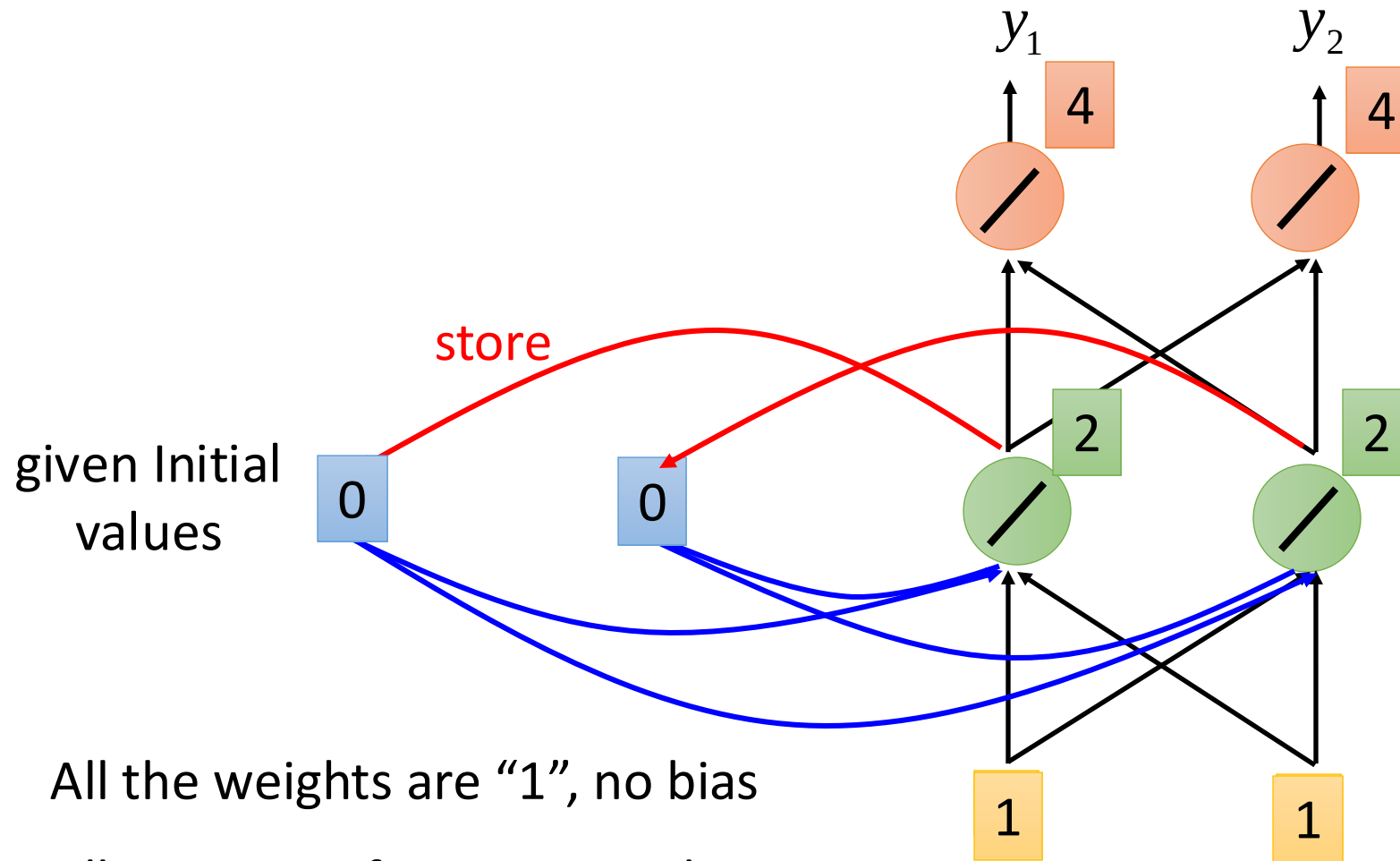
Example of RNN



Example

Input sequence: $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ $\begin{bmatrix} 2 \\ 2 \end{bmatrix}$

output sequence: $\begin{bmatrix} 4 \\ 4 \end{bmatrix}$



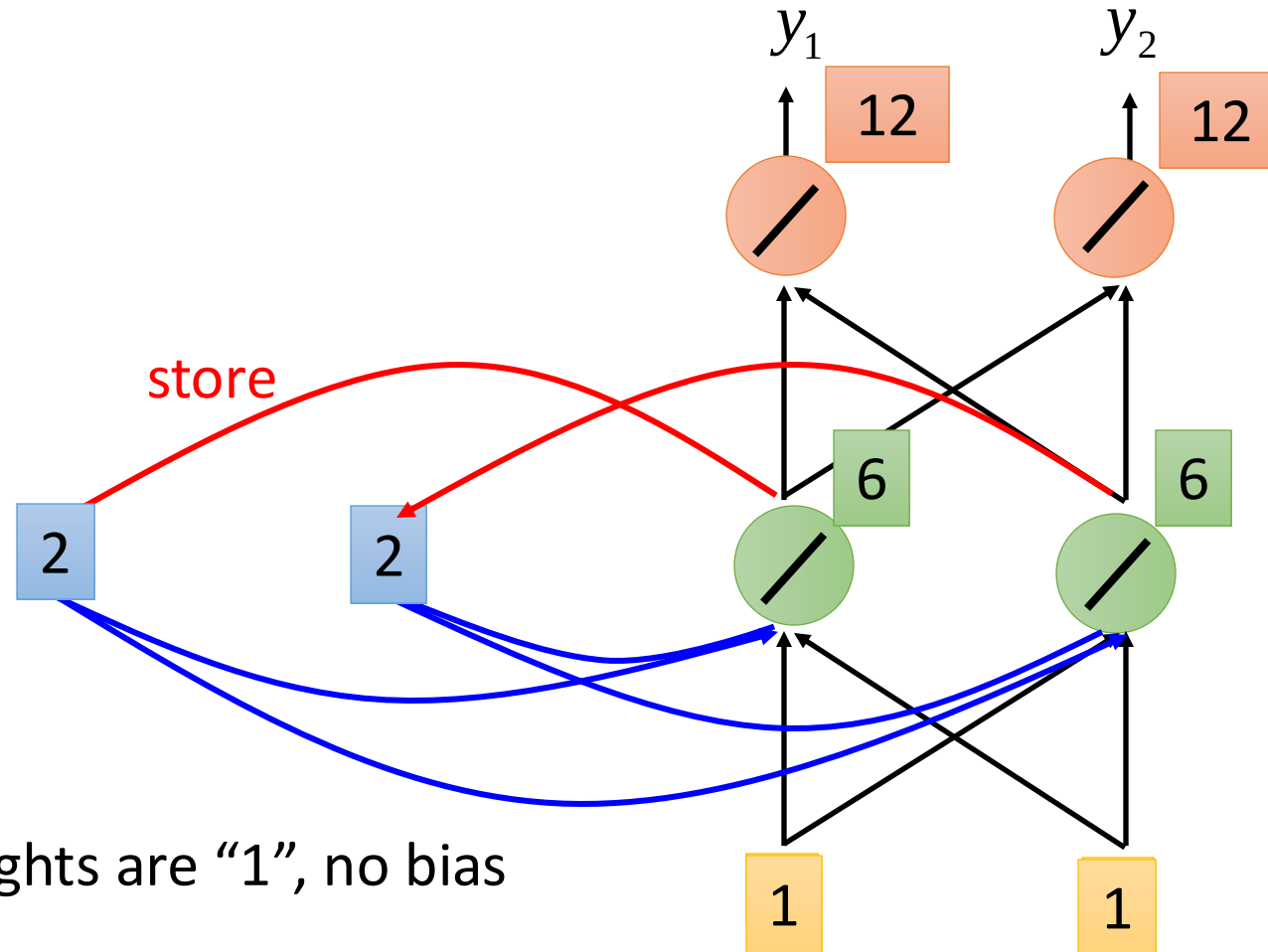
All the weights are "1", no bias

All activation functions are linear

Example

Input sequence: $\begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} \dots$

output sequence: $\begin{bmatrix} 4 \\ 4 \end{bmatrix} \begin{bmatrix} 12 \\ 12 \end{bmatrix}$



All the weights are "1", no bias

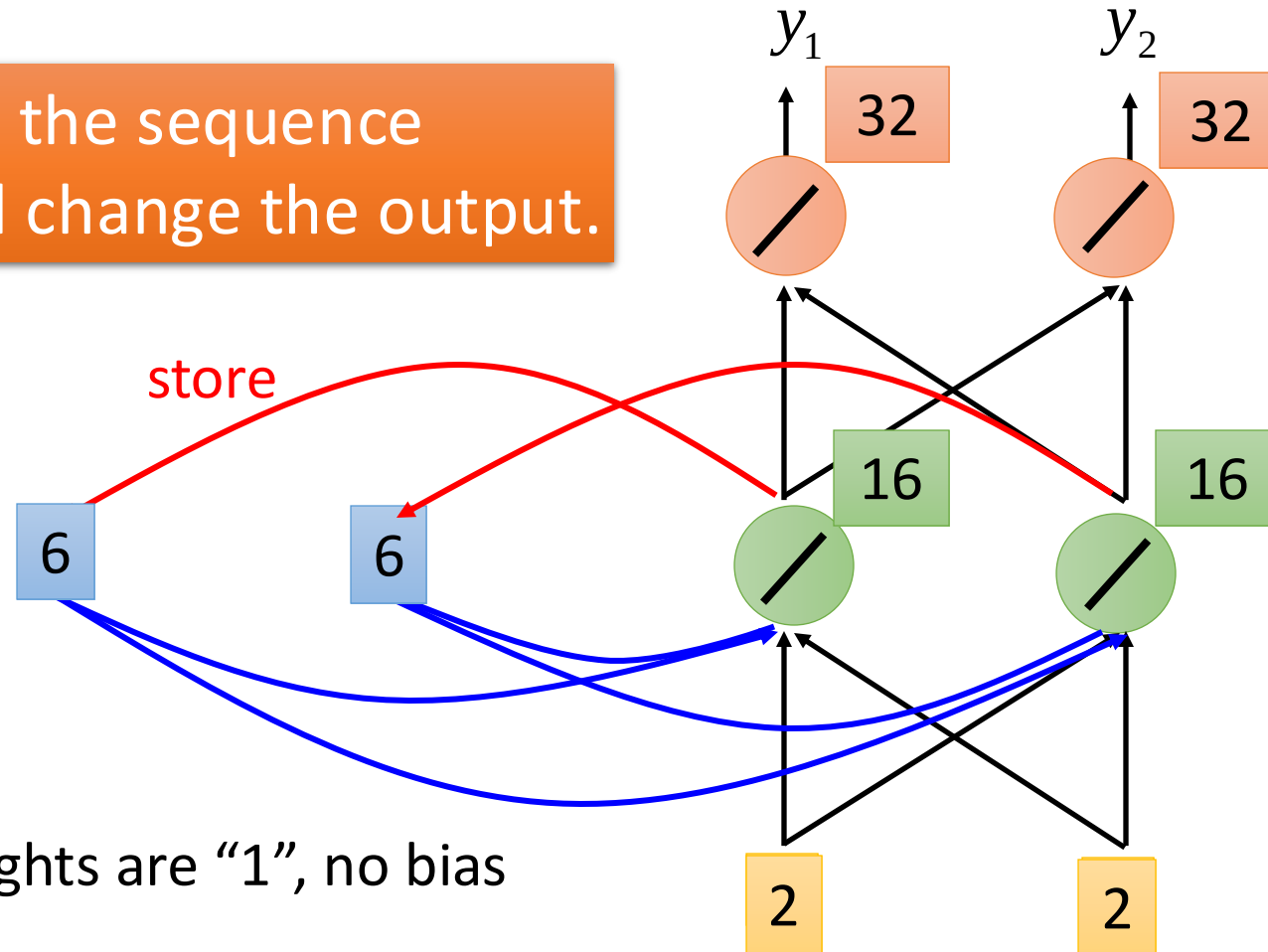
All activation functions are linear

Example

Input sequence: $\begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} \dots$

output sequence: $\begin{bmatrix} 4 \\ 4 \end{bmatrix} \begin{bmatrix} 12 \\ 12 \end{bmatrix} \begin{bmatrix} 32 \\ 32 \end{bmatrix}$

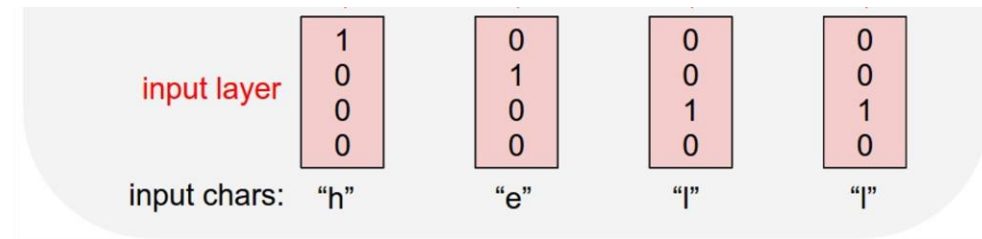
Changing the sequence order will change the output.



Example: language modeling

- A language model estimates a probability distribution of a linguistic unit (test, sentence, token, symbol)
 - It learn the rule of how people speak (or write)

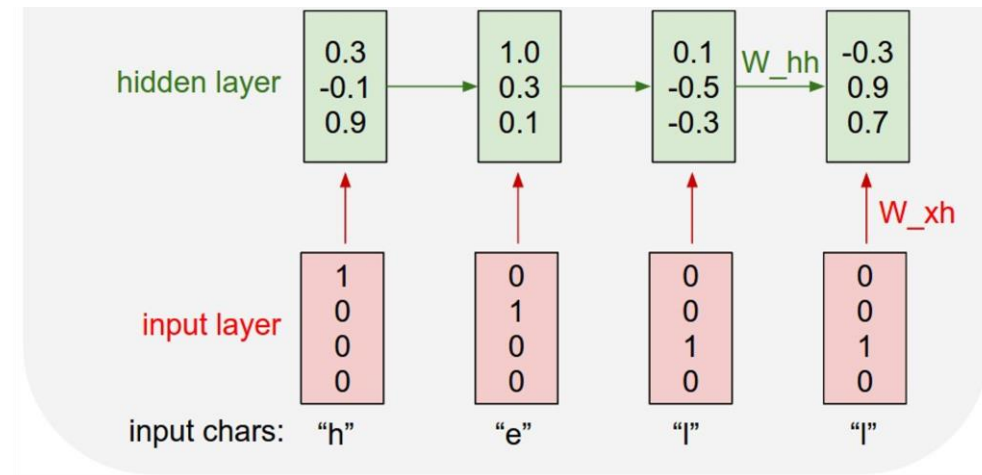
Training sequence: "hello"
Vocabulary: [h, e, l, o]



Example: language modeling

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: "hello"
Vocabulary: [h, e, l, o]

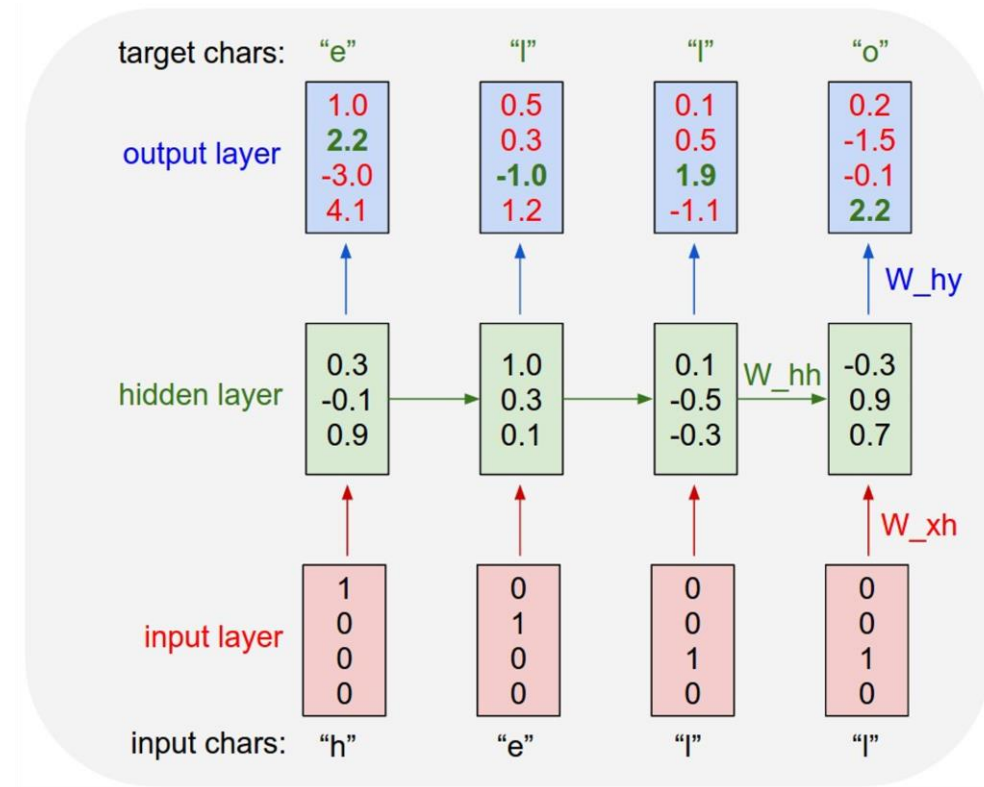


Example: language modeling

Given characters 1, 2, ..., t-1,
model predicts character t

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: "hello"
Vocabulary: [h, e, l, o]

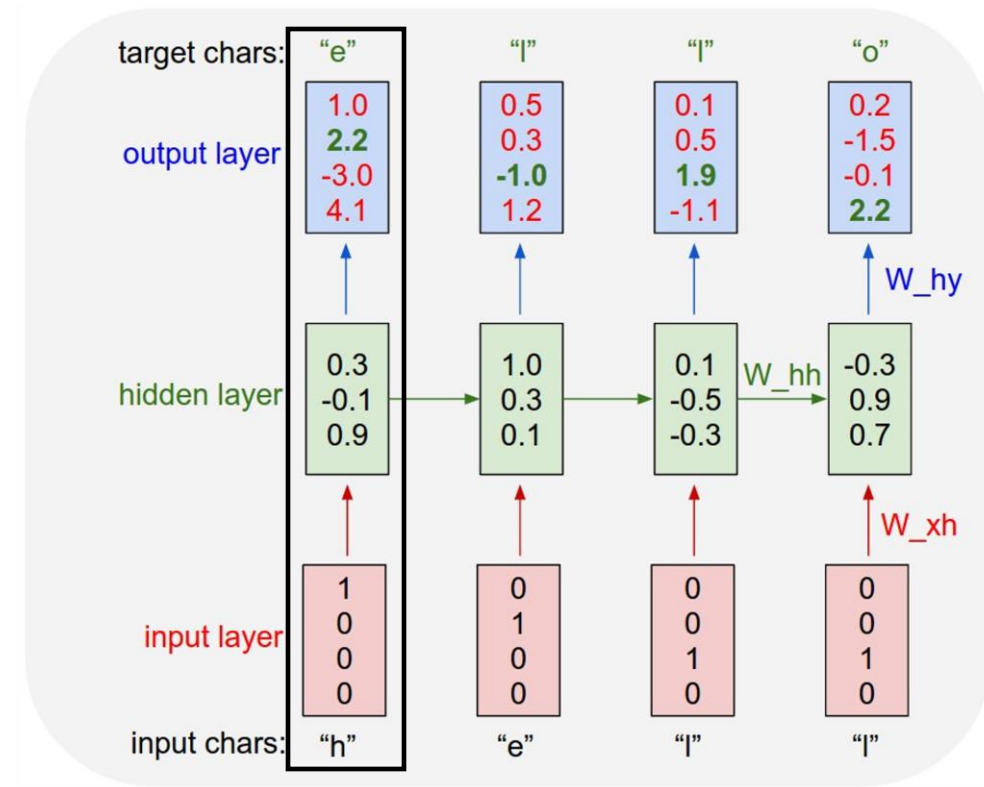


Example: language modeling

Given characters 1, 2, ..., t-1,
model predicts character t

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: "hello"
Vocabulary: [h, e, l, o]

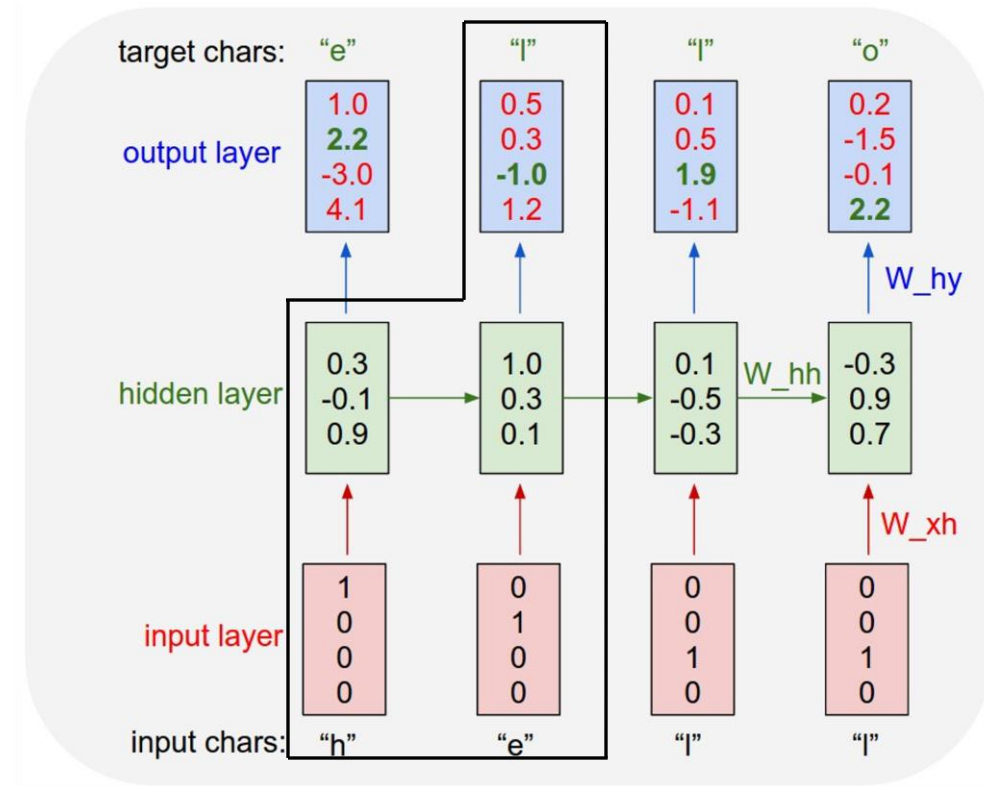


Example: language modeling

Given characters 1, 2, ..., t-1,
model predicts character t

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: "hello"
Vocabulary: [h, e, l, o]

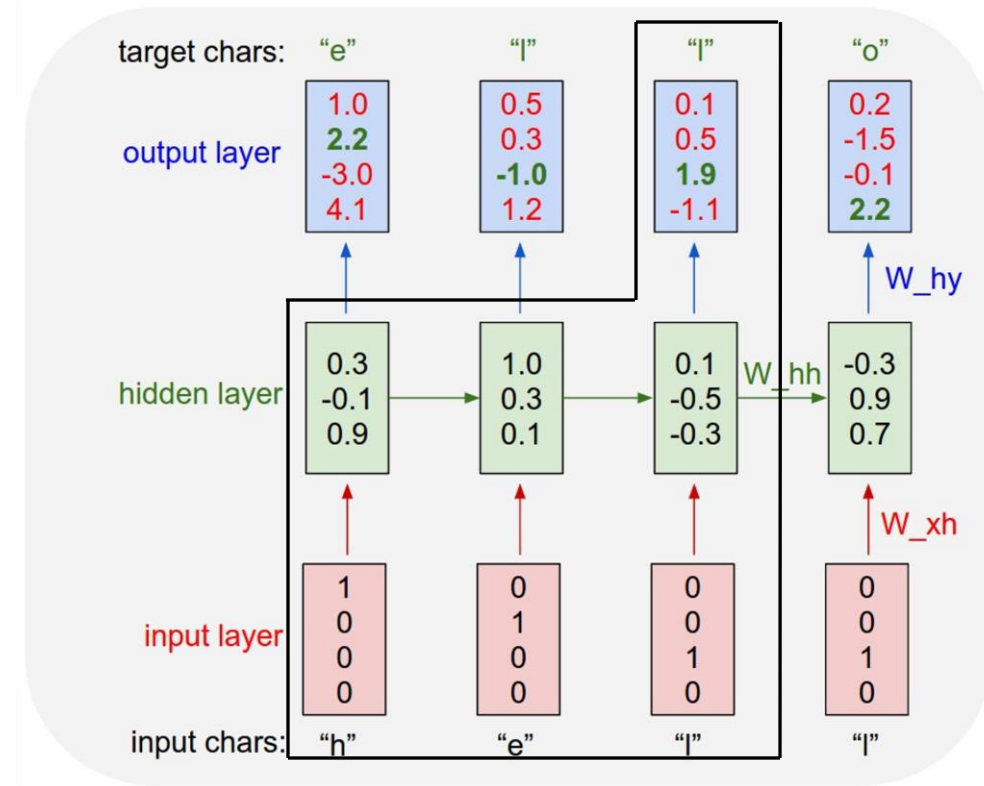


Example: language modeling

Given characters 1, 2, ..., t-1,
model predicts character t

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: "hello"
Vocabulary: [h, e, l, o]

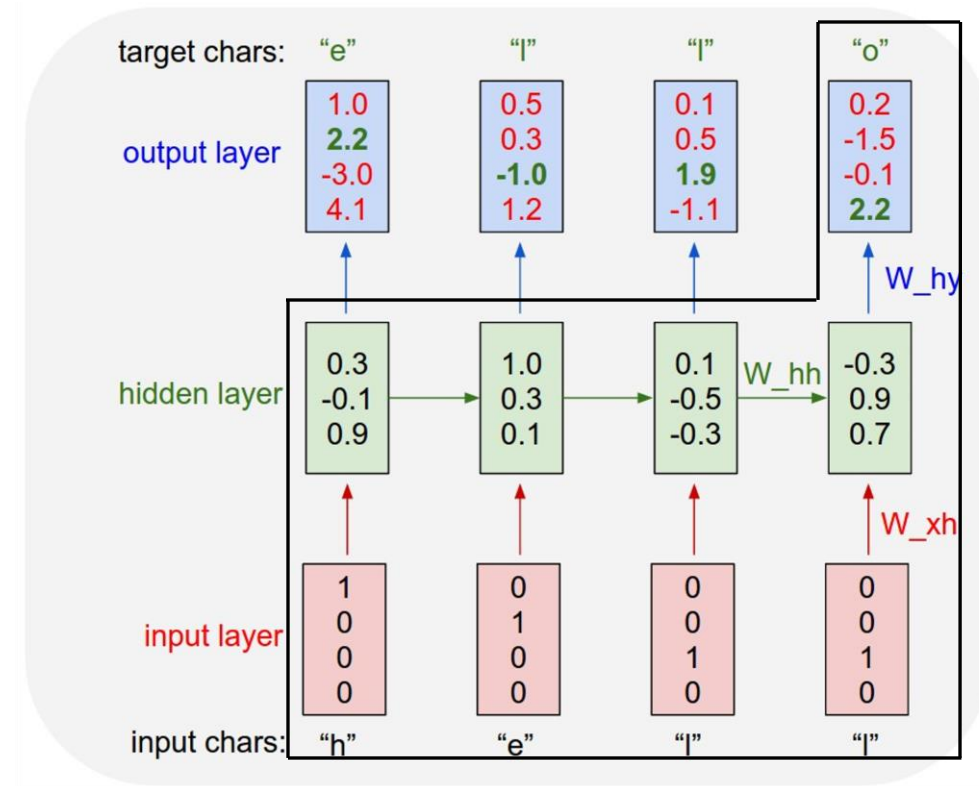


Example: Language Modeling

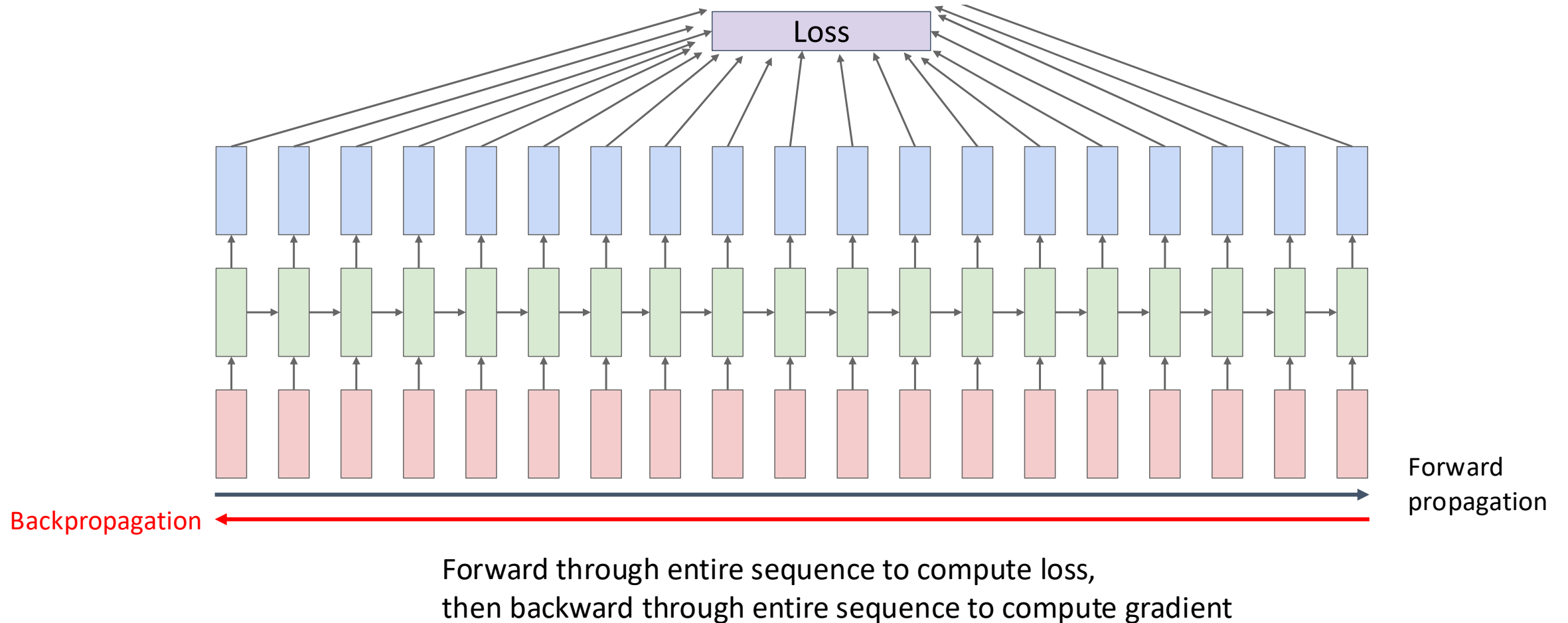
Given characters 1, 2, ..., t-1,
model predicts character t

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: "hello"
Vocabulary: [h, e, l, o]



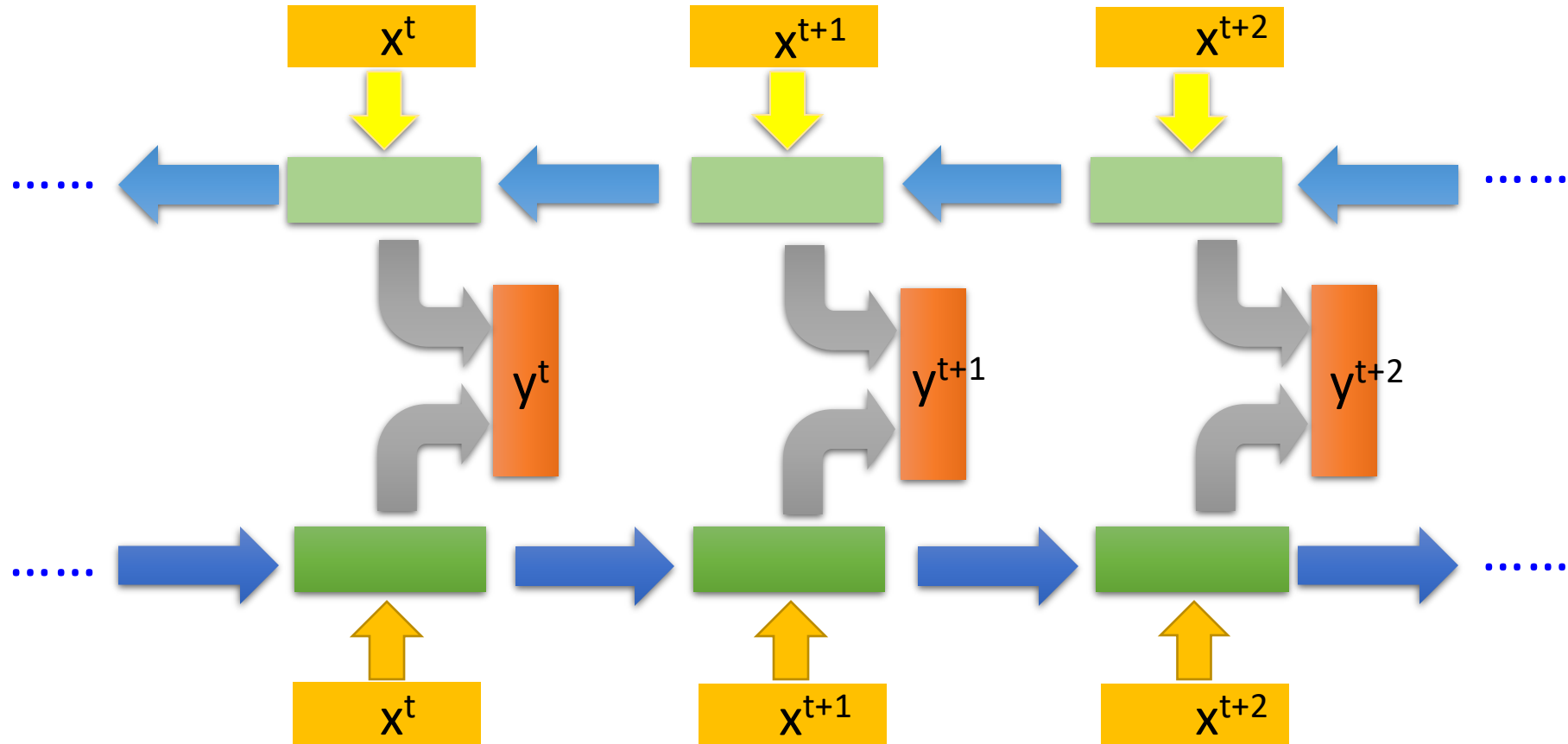
Backpropagation Through Time



Bidirectional RNN

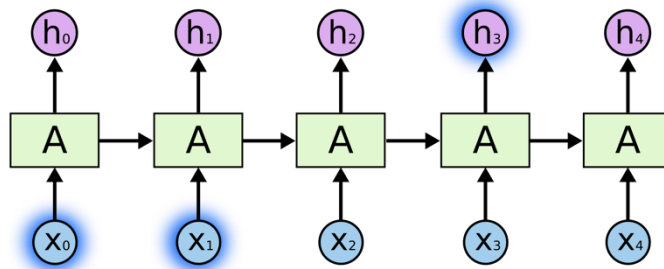
- Filling in the blank in a text sequence:
 - I am ____.
 - I am ____ hungry.
 - I am ____ hungry, and I can eat half a pig.
- The end of the phrase (if available) conveys significant information about which word to pick
- *Bidirectional RNNs*
 - Instead of running an RNN only in the forward mode starting from the first token, we **start another one from the last token** running from back to front.

Bidirectional RNN



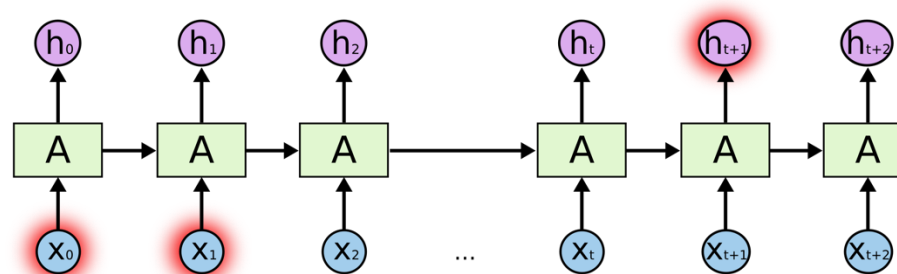
Problem of long-term dependencies

- RNNs are able to connect previous information to the present task



predict the last word in “the clouds are in the sky,”

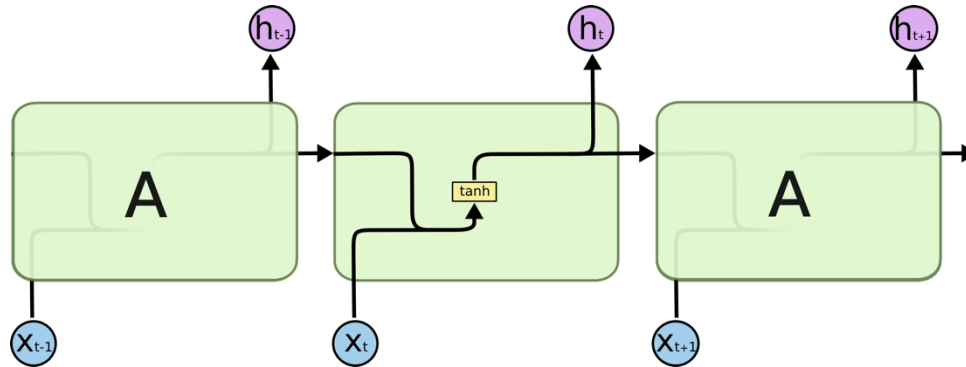
- Unfortunately, as that gap grows, RNNs become unable to learn to connect the information



predict the last word in the text “I grew up in France ... I speak fluent French.”

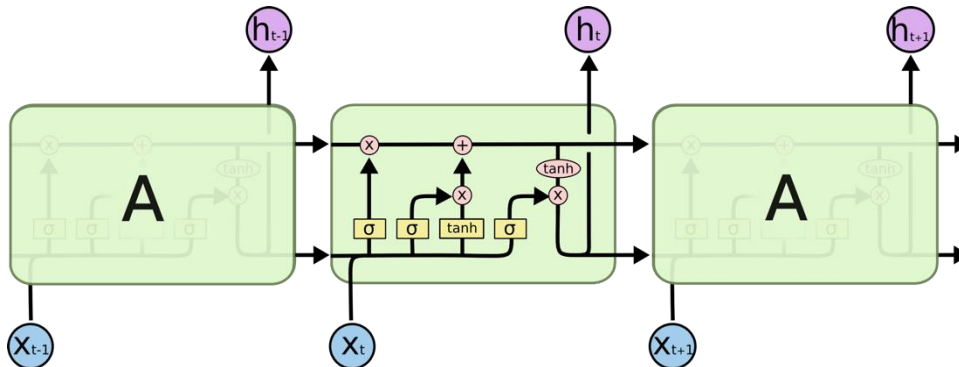
Long Short Term Memory (LSTM) networks

- Standard RNN



The repeating module in a standard RNN contains a single layer

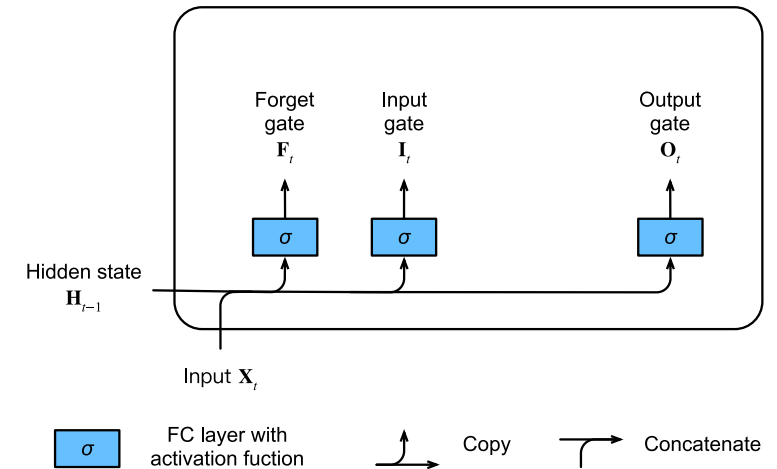
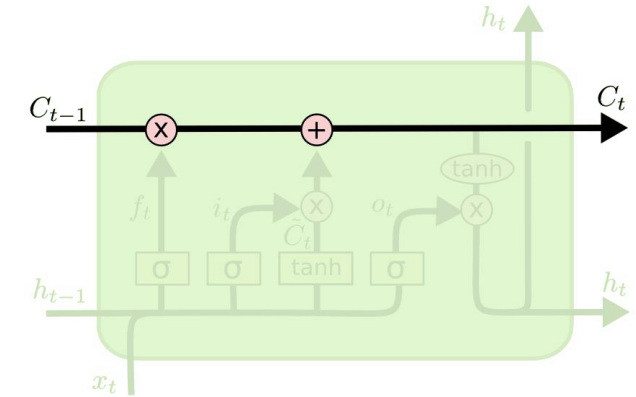
- LSTMs are explicitly designed to avoid the long-term dependency problem



The repeating module in an LSTM contains four interacting layers

Core idea behind LSTM

- Key to LSTMs is the *memory cell*
 - Has the same shape as the hidden state
 - Designed to record additional information
- To control the memory cell we need a number of gates; gates are a way to optionally let information through
 - *forget gate*: reset the content of the cell
 - *input gate*: decide when to read data into the cell. We refer to this as the
 - *output gate*: read out the entries from the cell

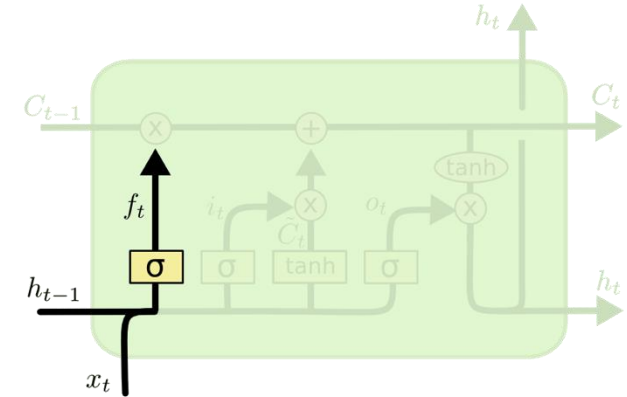


Step-by-step in LSTM

- What information to forget
- The first step in LSTM is to decide what information we're going to throw away from the cell state.
- The decision is made by a sigmoid layer called the **forget gate**
 - Input: previous state h_{t-1} and current input x_t
 - Output: a number between 0 and 1 applied on previous cell state C_{t-1}

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- Decision
 - $f_t = 1$ represents “completely keep”
 - $f_t = 0$ represents “completely forget”



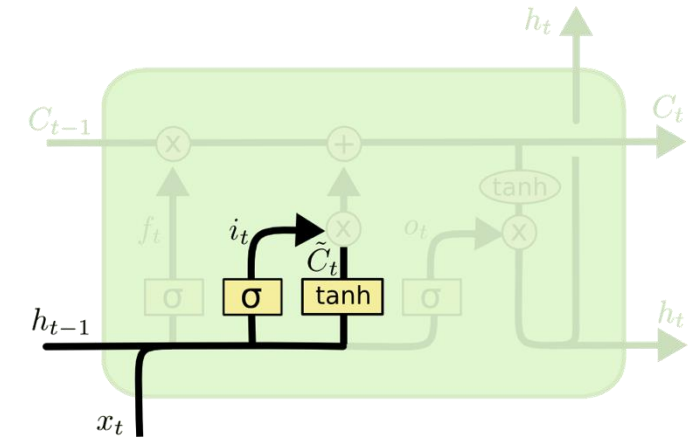
Step-by-step in LSTM

- What new information to store
- Decide what new information to store in the cell state
- First, sigmoid layer called **input gate** decides which values we'll update

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

- Second, tanh layer creates a new **candidate memory cell**
 - It could be added to the state.

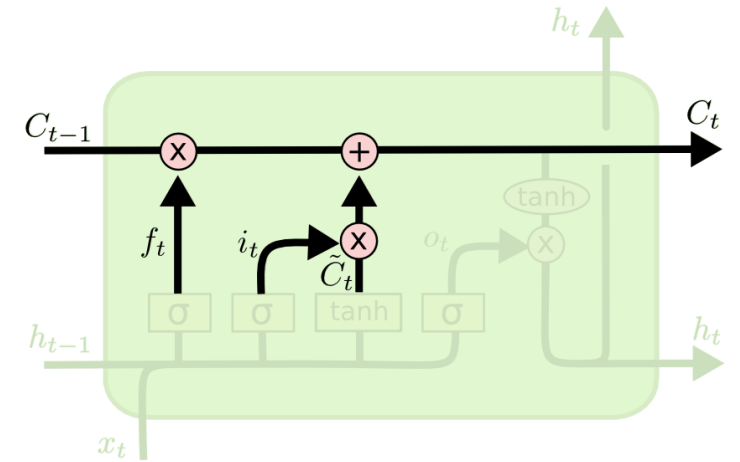
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$



Step-by-step in LSTM

- **Update the cell state**
- Update old cell state C_{t-1} to new cell state C_t
- Part 1: Multiply the old state by f_t ,
 - forgetting the things we decided to forget earlier.
- Part 2: Add $i_t \times \tilde{C}_t$
 - $\tilde{C}_t$ is the new candidate values
 - i_t decides how much we decided to update each state value.

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$



Step-by-step in LSTM

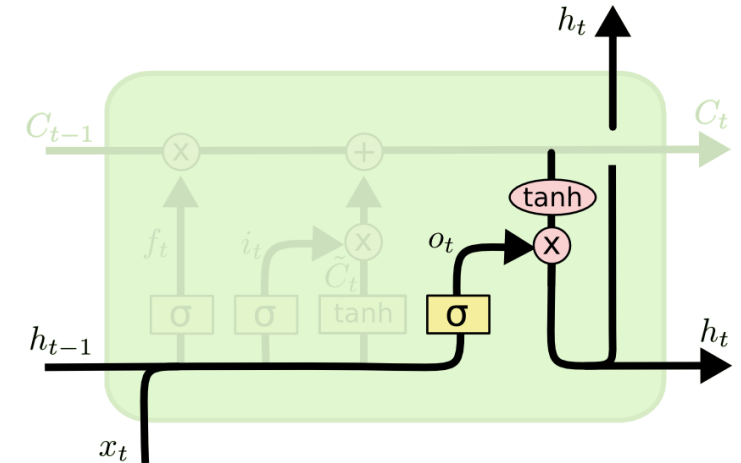
- What to output
- Goal: decide what to output for hidden state h_t
- Output o_t from the **output gate**

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

- Out put is the gated version of the tanh of the memory cell

$$h_t = o_t * \tanh(C_t)$$

- output gate close to 1: pass all memory information,
- output gate close to 0: we retain all the information only within the memory cell and perform no further processing



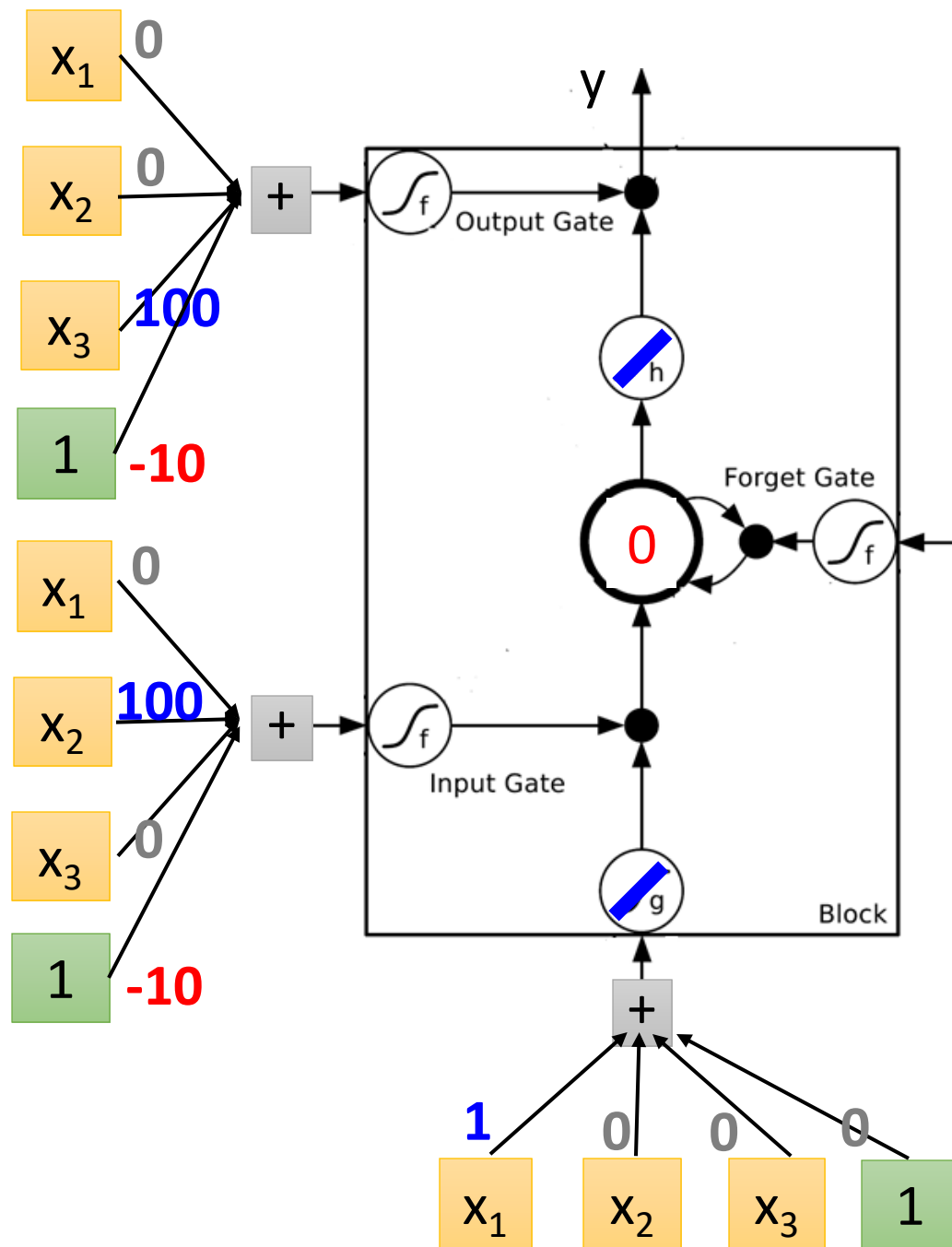
LSTM - Example

	0	0	3	3	7	7	7	0	6
x_1	1	3	2	4	2	1	3	6	1
x_2	0	1	0	1	0	0	-1	1	0
x_3	0	0	0	0	0	1	0	0	1
y	0	0	0	0	0	7	0	0	6

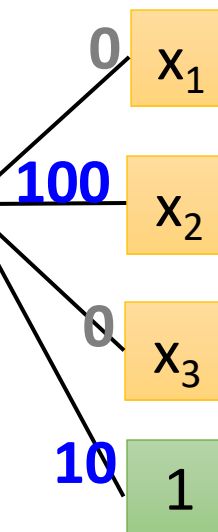
When $x_2 = 1$, add the numbers of x_1 into the memory

When $x_2 = -1$, reset the memory

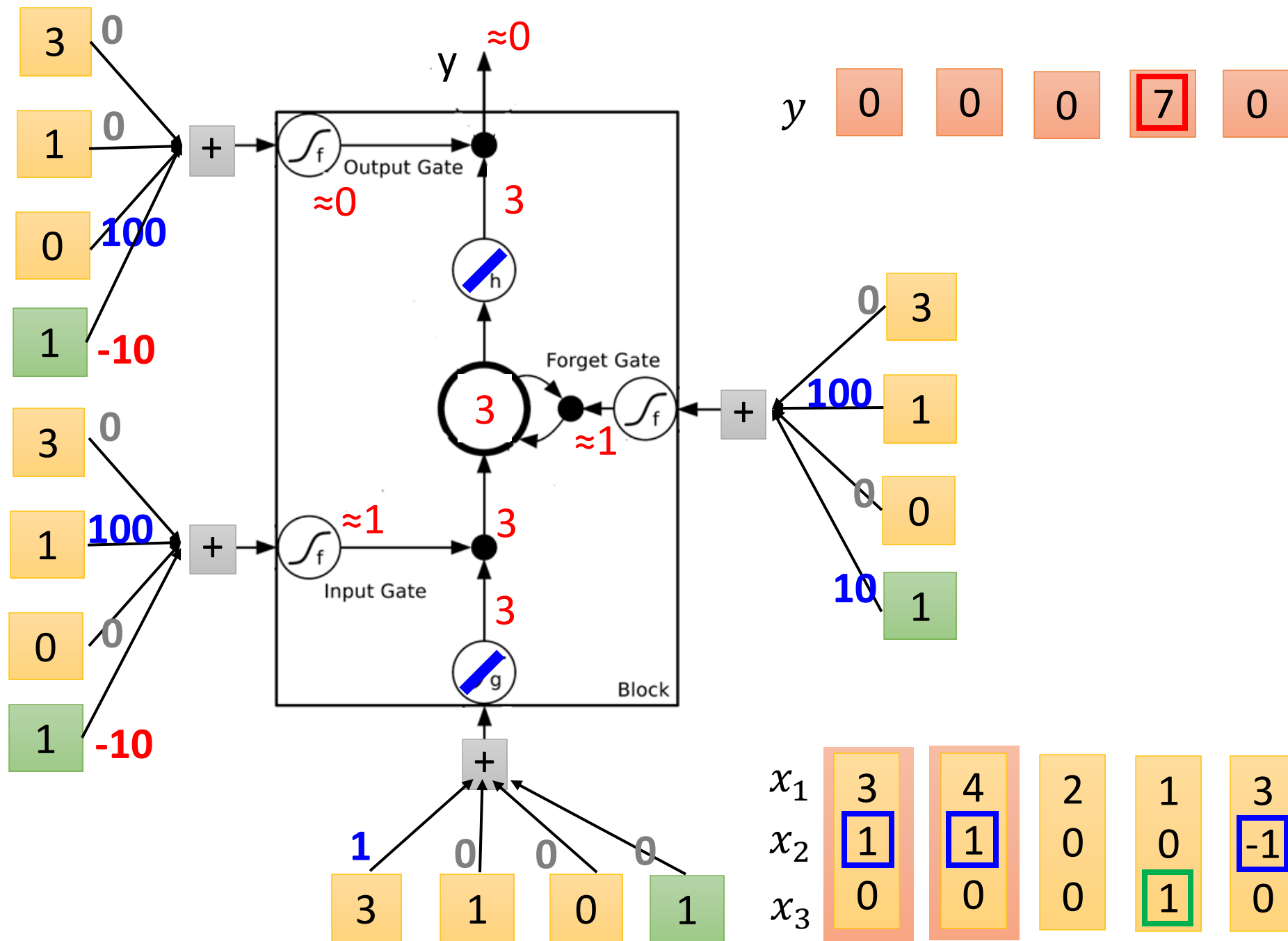
When $x_3 = 1$, output the number in the memory.

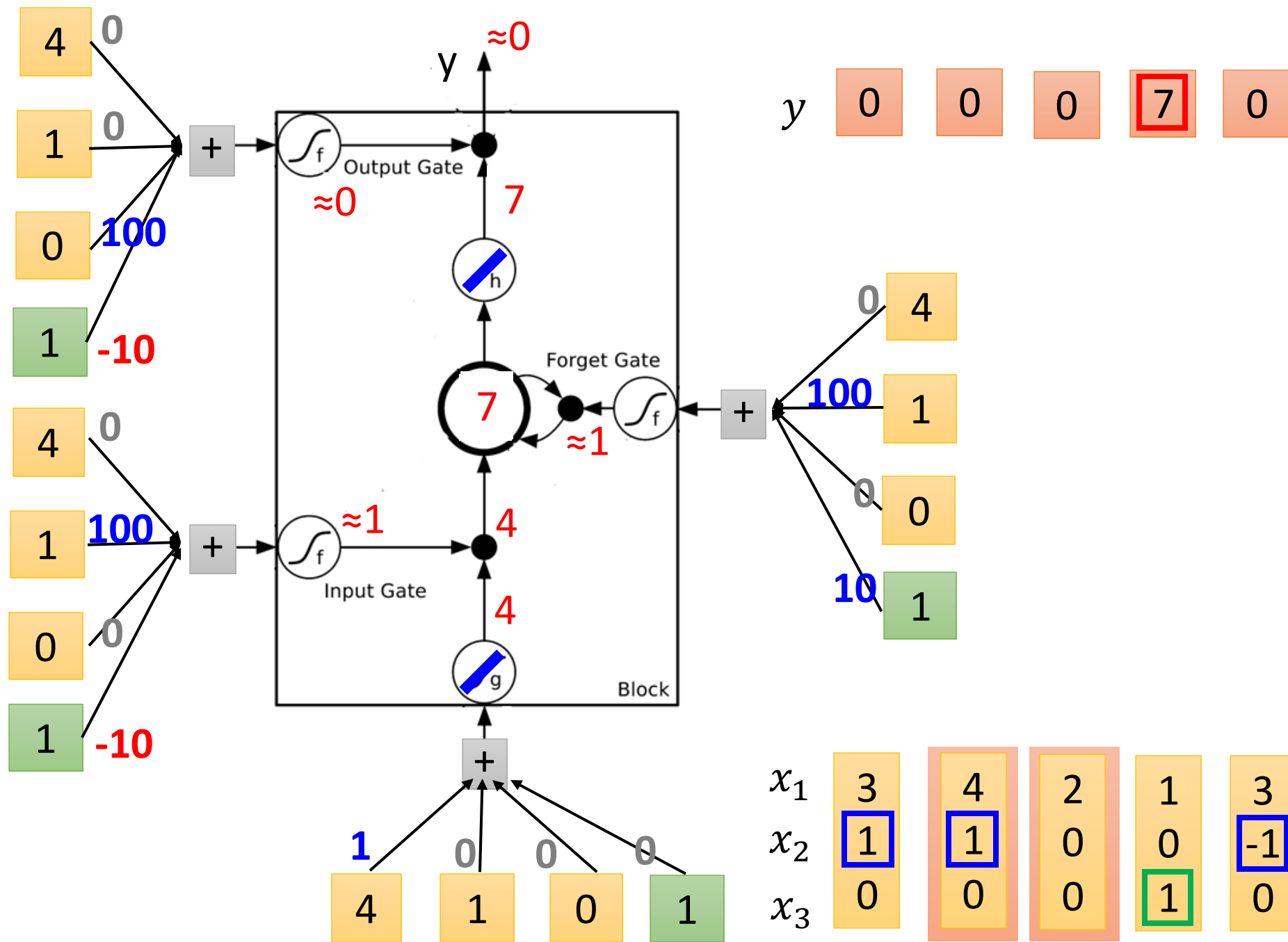


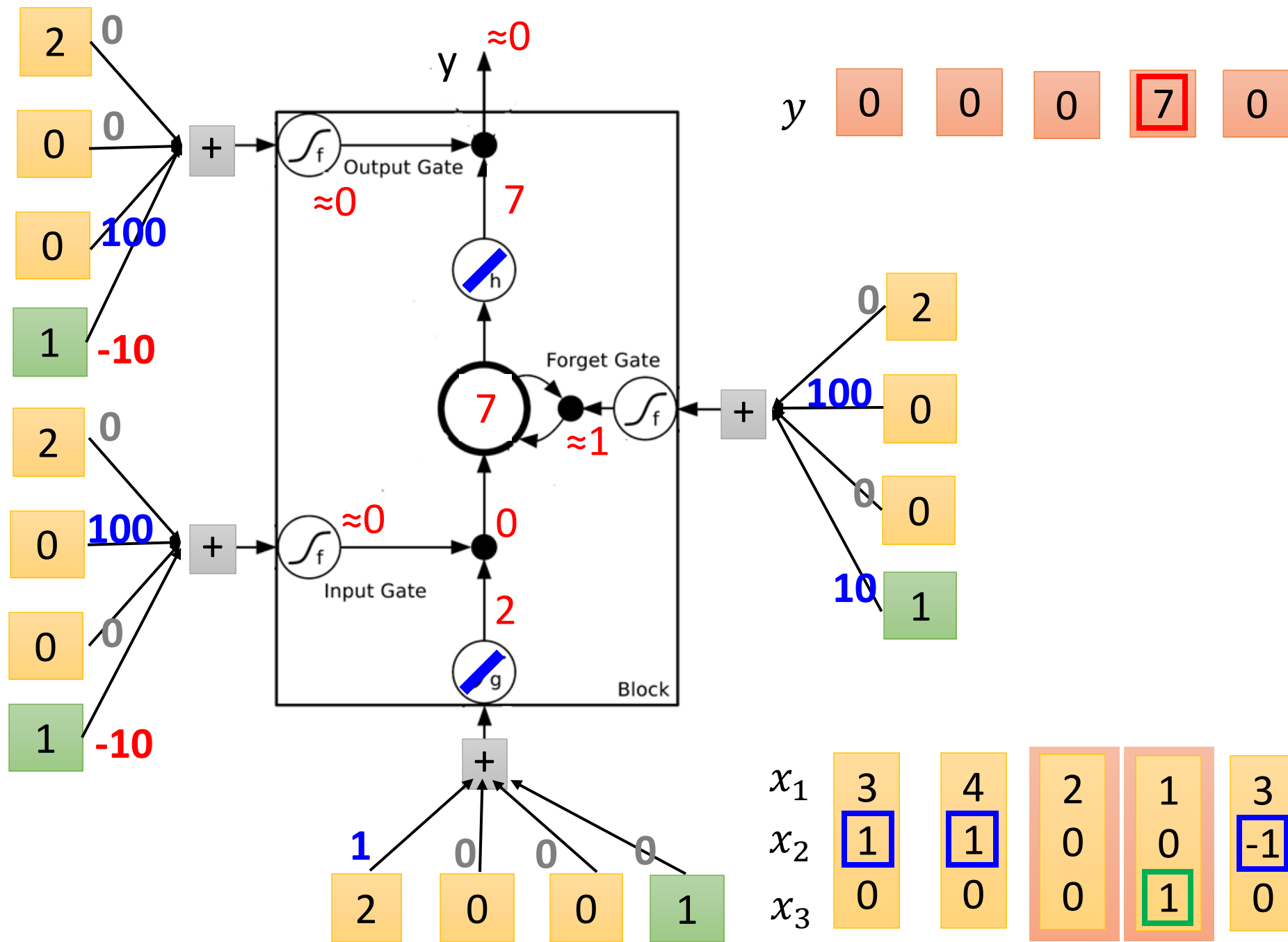
y 0 0 0 7 0

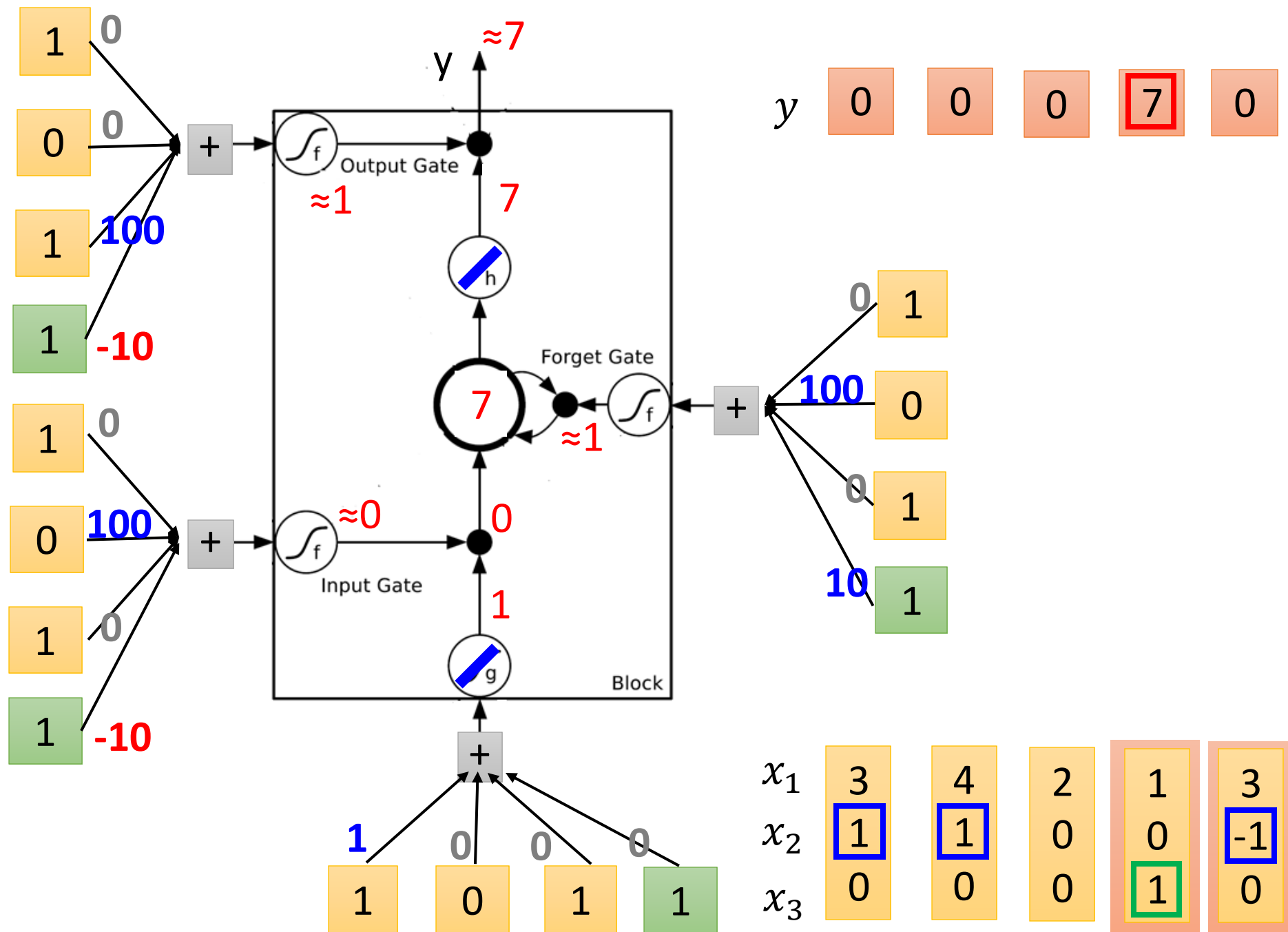


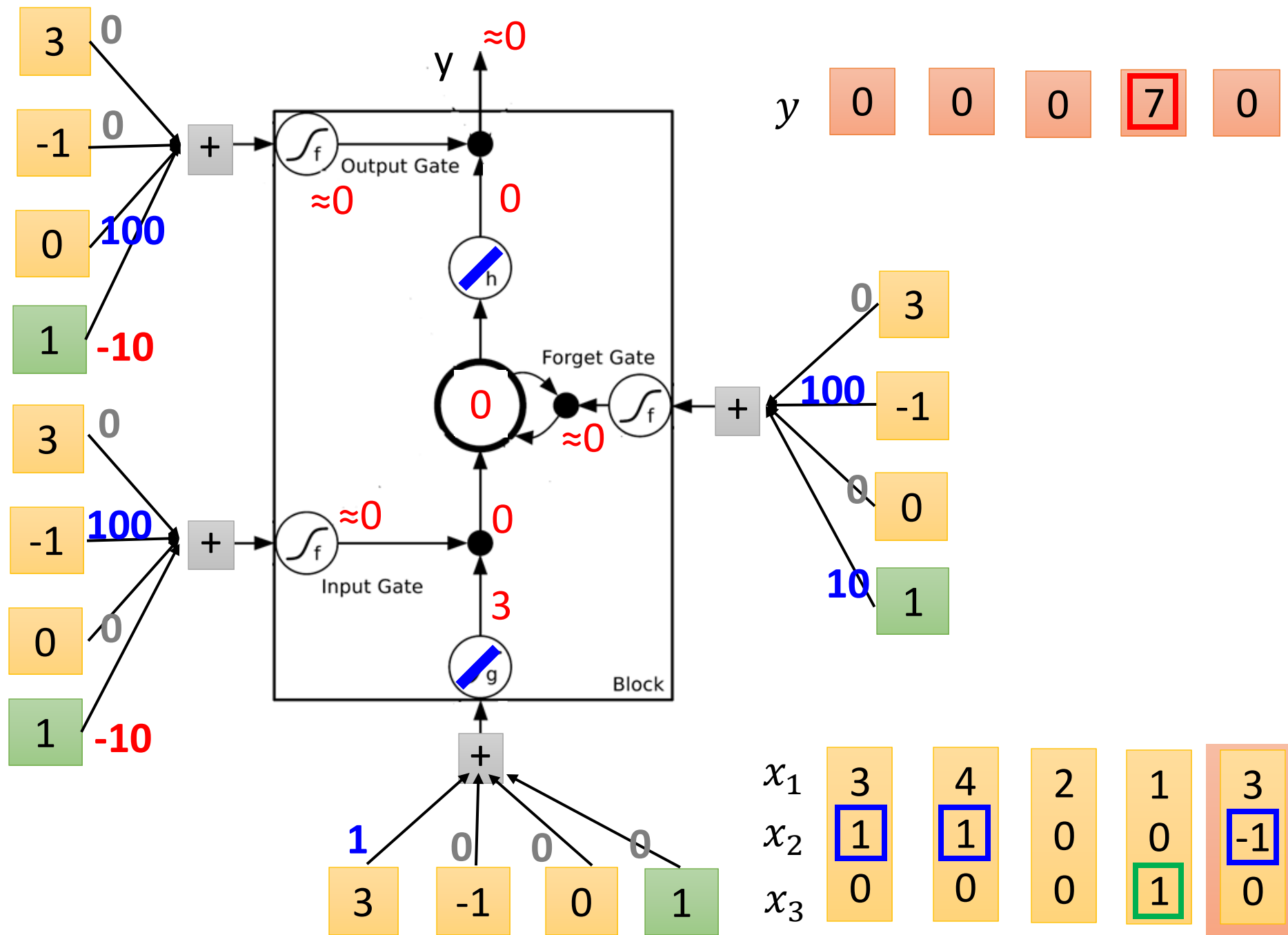
	3	4	2	1	3
x_1	3	4	2	1	3
x_2	1	1	0	0	-1
x_3	0	0	0	1	0



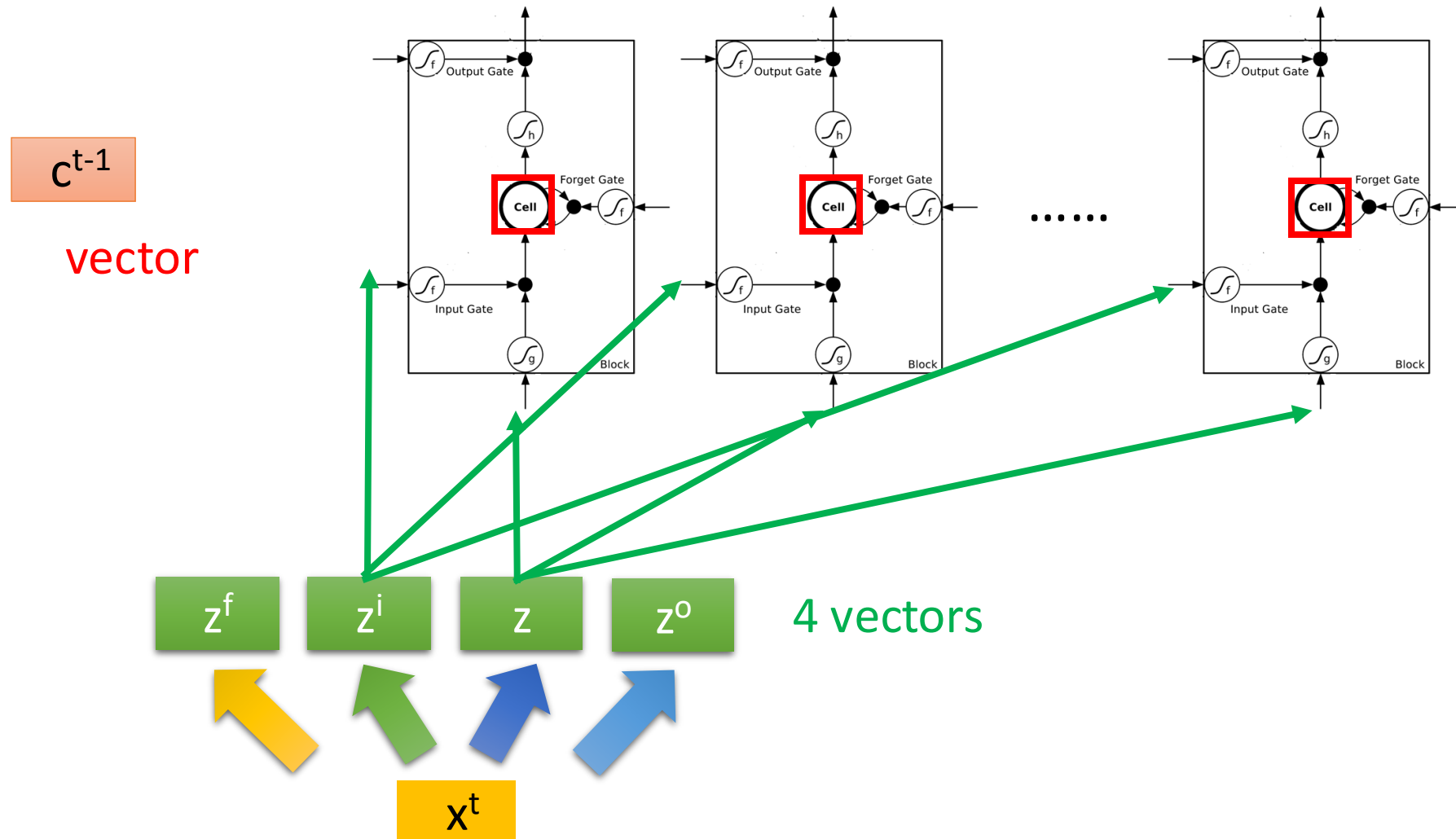




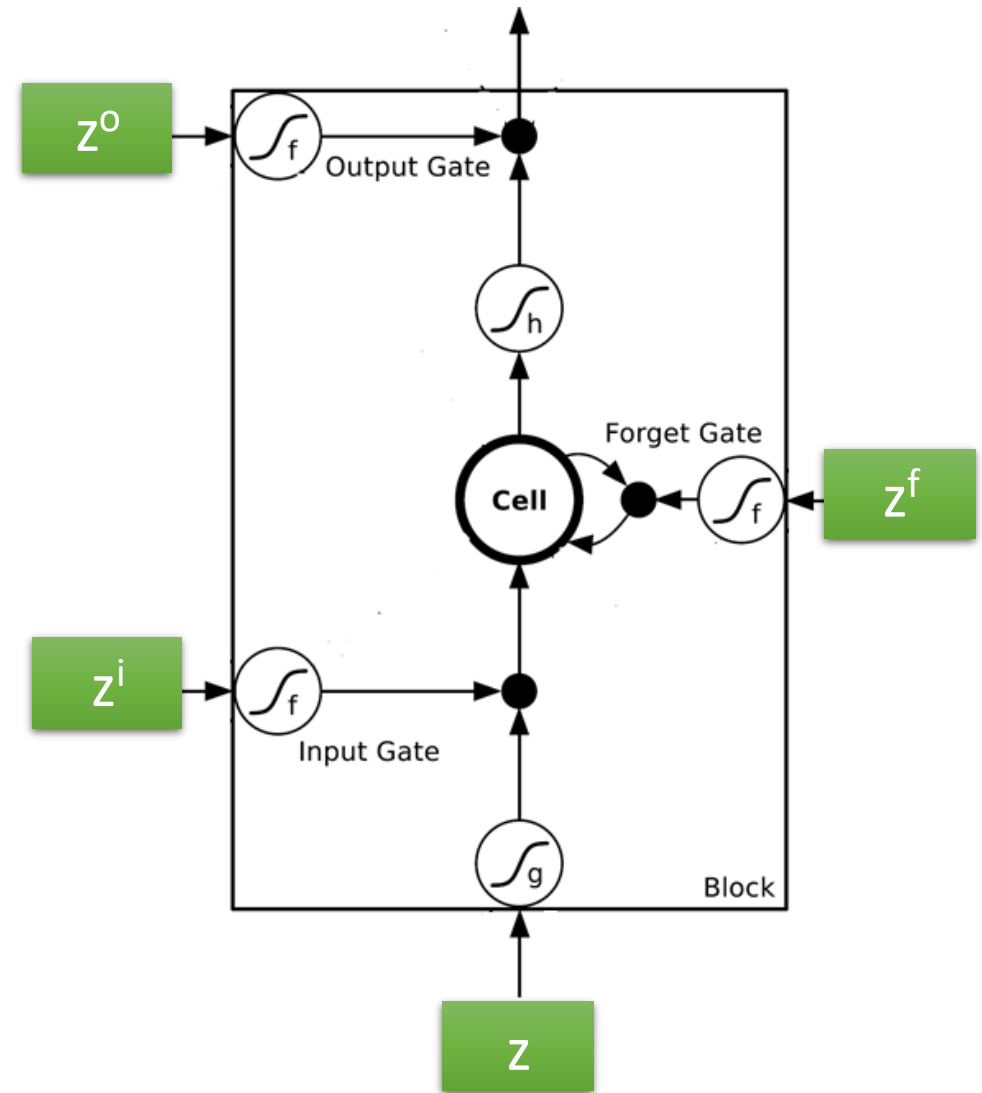
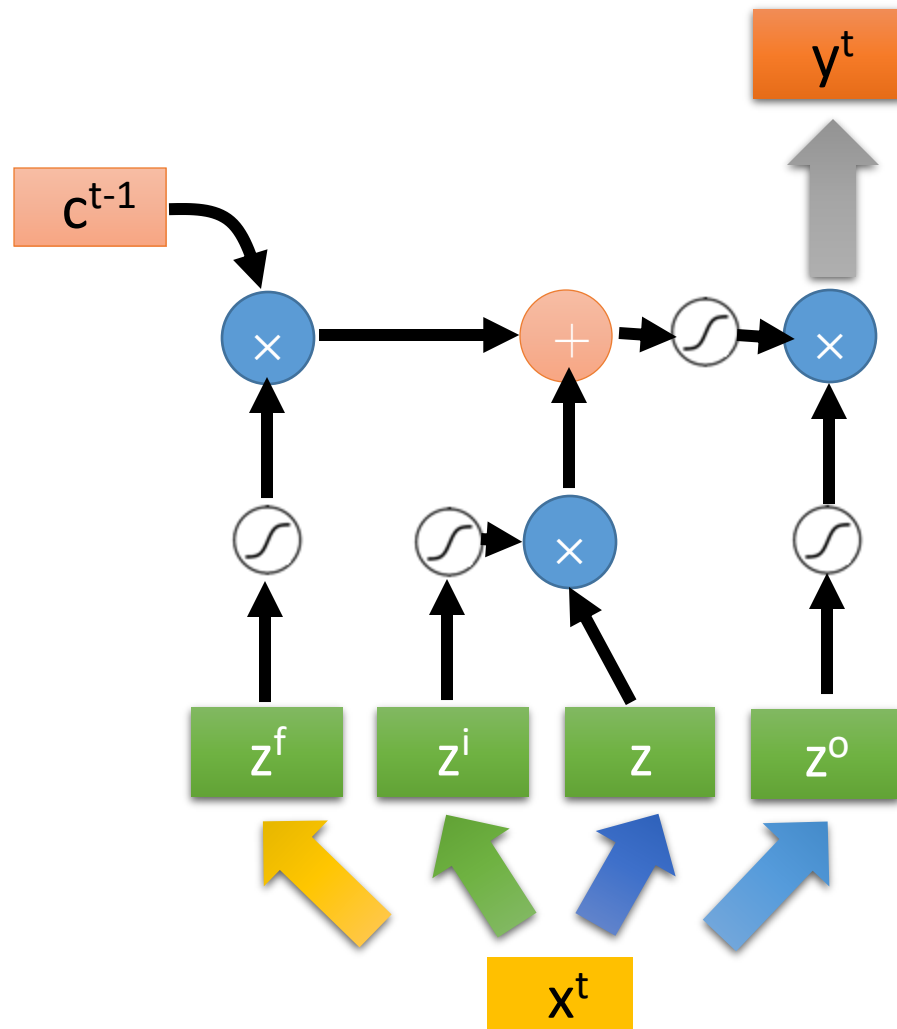




LSTM



LSTM



Gated Recurrent Unit (GRU)

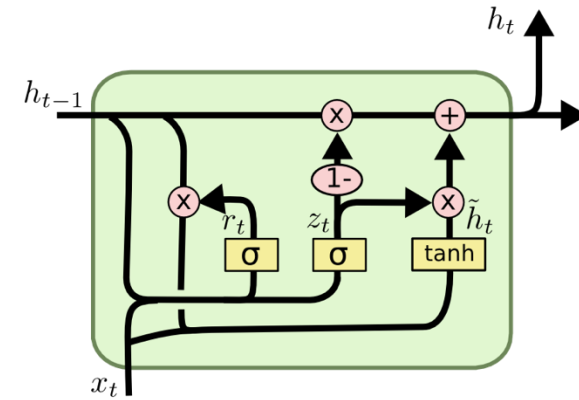
- GRU combines the forget and input gates into a single “*update gate*”
- GRU merges the cell state and hidden state
- GRU is *simpler* than standard LSTM models, and has been growing increasingly popular

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

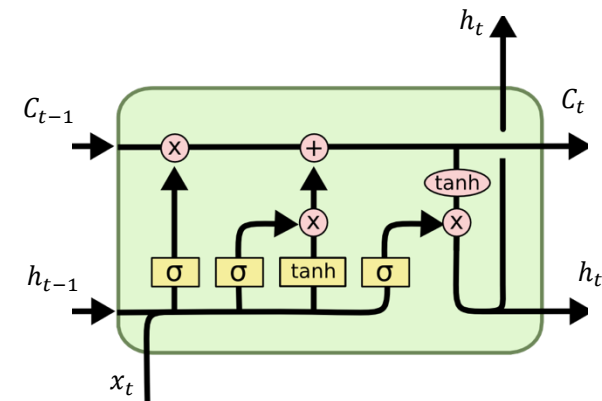
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$



GRU



LSTM

Summary

- RNNs allow a lot of flexibility in architecture design
- Vanilla RNNs are simple but don't work very well
- Common to use LSTM and GRU: additive interactions improve gradient flow
- RNNs can go deep
- Wide applications in NLP